KOCAELİ ÜNİVERSİTESİ MÜHENDİSLİK FAKÜLTESİ

IoT WALL OUTLET WİTH RASPERRY Pİ and ESP8266

MÜHENDİSLİK TASARIMI – 2

PINAR AYER

220208024

Danışman: Doçent Doktor Öğretim Üyesi Oğuzhan Karahan

Kocaeli, 2026

İÇİNDEKİLER

İÇİNDEKİLER i
ŞEKİLLER DİZİNİ ii

1. GİRİŞ 1
2. LİTERATÜR ARAŞTIRMASI 2
3. PROJEDE KULLANILAN YAZILIM VE DONANIM TEKNOLOJİLERİ 4
   3.1. Yazılım Teknolojileri 4
   3.1.1. Arduino IDE 4
   3.1.2. C/C++ Programlama Dili 5
   3.1.3. MQTT Protokolü 5
   3.1.4. Mosquitto MQTT Broker 5
   3.1.5. Node-RED 6
   3.1.6. Node-RED Dashboard 6
   3.1.7. JSON Veri Yapısı 7
   3.1.8. IoT MQTT Panel Mobil Uygulaması 7
   3.1.9. ThingSpeak 7

   3.2. Donanım Teknolojileri 8
   3.2.1. ESP32 Mikrodenetleyici 8
   3.2.2. Röle Modülü 9
   3.2.3. IR Alıcı Modülü 9
   3.2.4. IR Uzaktan Kumanda 10
   3.2.5. Buton 10
   3.2.6. Raspberry Pi 11
   3.2.7. DHT11 Sıcaklık ve Nem Sensörü 11
   3.2.8. LDR Sensör Modülü 12
   3.2.9. 5V DC Fan 12
   3.2.10. OLED Ekran 12

4. NESNELERİN İNTERNETİ (IoT) NEDİR? 14
5. PROJENİN GERÇEKLENMESİ 15
   5.1. Giriş 15

   5.2. Donanım Tasarımı 15
   5.2.1. Sistem Donanım Mimarisi 15
   5.2.2. ESP32 Pin Bağlantı Planı 16
   5.2.3. Pertinaks Üzerindeki Donanım Yerleşimi 16
   5.2.4. Güç Beslemesi ve Kablo Düzeni 18
   5.2.5. Kutulama Tasarımı ve Donanım Yerleşiminin Değerlendirilmesi 19

   5.3. Yazılım Tasarımı 23
   5.3.1. MQTT Mesaj İşleme Yapısı 24
   5.3.2. Wi-Fi ve MQTT Bağlantı Yapısı 24
   5.3.3. Ana Döngü, Buton ve IR Kumanda Kontrolü 25
   5.3.4. ESP32 Yazılımının Genel Değerlendirmesi 26
   5.3.5. Node-RED Akış Tasarımı 26

5.4. Deneysel Sonuçlar 29

6. SONUÇLAR VE ÖNERİLER 35

KAYNAKÇA 37

ŞEKİLLER DİZİNİ

Şekil 3.1. ESP32 8
Şekil 3.2. Röle Modülü 9
Şekil 3.3. IR Alıcı Modülü 9
Şekil 3.4. IR Uzaktan Kumanda 10
Şekil 3.5. Buton 10
Şekil 3.6. Raspberry Pi 11
Şekil 3.7. DHT11 Sıcaklık ve Nem Sensörü 11
Şekil 3.8. LDR Sensör Modülü 12
Şekil 3.9. 5V DC Fan 12
Şekil 3.10. OLED Ekran 13

Şekil 5.1. ESP32 Pin Bağlantıları 16
Şekil 5.2. Pertinaks Görünümü 17
Şekil 5.3. Projenin İç Görüntüsü 20
Şekil 5.4. Projenin Dış Görüntüsü 21
Şekil 5.5. MQTT Mesaj İşleme Yapısı 24
Şekil 5.6. Wi-Fi ve MQTT Bağlantı Yapısı 25
Şekil 5.7. Ana Döngü, Buton ve IR Kumanda Kontrolü 25
Şekil 5.8. DHT11 Sıcaklık ve Nem Verisi Node-RED Akışı 26
Şekil 5.9. LDR Işık Durumu Node-RED Akışı 27
Şekil 5.10. Fan Kontrol Node-RED Akışı 27
Şekil 5.11. Lamba Kontrol Node-RED Akışı 27
Şekil 5.12. ThingSpeak Veri Gönderme Node-RED Akışı 28
Şekil 5.13. Telegram Bildirim Node-RED Akışı 28
Şekil 5.14. Node-RED Dashboard Üzerinden Lamba Kontrolü 29
Şekil 5.15. Node-RED Dashboard Üzerinden Fan Kontrolü 30
Şekil 5.16. Node-RED Dashboard Üzerinde Sıcaklık ve Nem Verilerinin Gösterimi 30
Şekil 5.17. Node-RED Dashboard Üzerinde LDR Işık Durumu Gösterimi 31
Şekil 5.18. ThingSpeak Üzerinde Lamba, Fan, Sıcaklık ve Nem Verilerinin İzlenmesi 31
Şekil 5.19. ThingSpeak Üzerinde Işık Seviyesi Verisinin İzlenmesi 32
Şekil 5.20. Telegram Üzerinden Lamba ve Fan Durum Bildirimleri 32
Şekil 5.21. Lamba Açık Durumunda Sistemin Fiziksel Görünümü 33
Şekil 5.22. Lamba Kapalı Durumunda Sistemin Fiziksel Görünümü 34

1.GİRİŞ

Günümüzde Nesnelerin İnterneti (IoT) teknolojilerinin hızlı gelişimi, günlük yaşamda kullanılan elektrikli ve elektronik cihazların internet üzerinden izlenebilmesini ve kontrol edilebilmesini mümkün hâle getirmiştir. Özellikle akıllı ev sistemlerinin yaygınlaşmasıyla birlikte kullanıcıların cihazlara uzaktan erişebilmesi, ortam koşullarını takip edebilmesi ve farklı kontrol yöntemlerini bir arada kullanabilmesi önemli bir ihtiyaç hâline gelmiştir. Bu doğrultuda geliştirilen IoT tabanlı sistemler; kullanıcı konforunun artırılması, enerji tüketiminin daha verimli yönetilmesi ve cihazların daha esnek bir yapıda kontrol edilebilmesi açısından önemli avantajlar sağlamaktadır.

Bu proje kapsamında IoT tabanlı akıllı priz ve ortam izleme sistemi geliştirilmiştir. Sistem, ESP32 mikrodenetleyicisi temel alınarak tasarlanmış olup lamba ve fan gibi elektriksel yüklerin röle modülleri aracılığıyla kontrol edilmesini sağlamaktadır. Bunun yanında sıcaklık, nem ve ortam aydınlık seviyesi gibi çevresel veriler sensörler yardımıyla ölçülerek sistem üzerinden izlenebilmektedir. Böylece proje yalnızca cihaz kontrolüne yönelik bir yapı sunmakla kalmayıp aynı zamanda ortam koşullarının takip edilmesine de olanak sağlamaktadır.

Geliştirilen sistemde kullanıcı etkileşimini artırmak amacıyla birden fazla kontrol yöntemi kullanılmıştır. Lamba ve fan; fiziksel buton, kızılötesi (IR) uzaktan kumanda, MQTT tabanlı mobil uygulama ve Node-RED Dashboard arayüzü üzerinden kontrol edilebilmektedir. Bu sayede kullanıcı, farklı kullanım senaryolarına uygun olarak sistemi hem yerel hem de kablosuz olarak yönetebilmektedir. Kontrol işlemlerinde meydana gelen durum değişiklikleri sistem tarafından takip edilmekte ve ilgili çıkış birimlerine aktarılmaktadır.

Projenin haberleşme altyapısında Wi-Fi teknolojisi ve MQTT protokolü kullanılmıştır. MQTT protokolünün yayınla-abone ol mimarisi sayesinde ESP32, Raspberry Pi üzerinde çalışan MQTT broker ve Node-RED arasında hızlı ve düşük gecikmeli bir veri alışverişi sağlanmıştır. Raspberry Pi, sistemde merkezi haberleşme ve kontrol birimi olarak görev yapmakta; Node-RED ise hem kontrol arayüzünün oluşturulmasında hem de verilerin işlenmesinde kullanılmaktadır. Ayrıca ThingSpeak platformu aracılığıyla sıcaklık, nem, ışık seviyesi ve cihaz durumları gibi verilerin izlenmesi sağlanmıştır.

Sistemde kullanılan OLED ekran sayesinde cihazların çalışma durumları ve sensörlerden elde edilen temel veriler kullanıcıya yerel olarak gösterilebilmektedir. Böylece kullanıcı, yalnızca internet tabanlı arayüzlerden değil, doğrudan sistem üzerindeki ekran aracılığıyla da anlık bilgilere erişebilmektedir. Bu yapı, sistemin kullanım kolaylığını artırmakta ve arıza ya da bağlantı durumlarının daha rahat takip edilmesine katkı sağlamaktadır.

Bu raporun amacı, geliştirilen IoT tabanlı akıllı priz ve ortam izleme sisteminin tasarım, kurulum ve test süreçlerini teknik ve akademik bir bakış açısıyla açıklamaktır. Rapor kapsamında kullanılan donanım ve yazılım teknolojileri, sistemin haberleşme yapısı, kontrol yöntemleri, sensör entegrasyonları ve elde edilen deneysel sonuçlar ayrıntılı olarak ele alınacaktır. Bu yönüyle proje, mühendislik eğitimi kapsamında teorik bilgilerin gerçek bir uygulama üzerinde kullanılması ve IoT tabanlı sistem tasarımı konusunda uygulamalı bir çalışma ortaya konulması açısından önem taşımaktadır.

2.LİTERATÜR ARAŞTIRMASI

Son yıllarda Nesnelerin İnterneti (IoT) teknolojilerinin gelişmesiyle birlikte, günlük yaşamda kullanılan elektrikli ve elektronik cihazların internet üzerinden izlenmesi ve kontrol edilmesine yönelik çalışmalar önemli ölçüde artmıştır. Özellikle akıllı ev sistemleri kapsamında geliştirilen uygulamalar; aydınlatma kontrolü, priz kontrolü, ortam izleme, enerji yönetimi ve güvenlik gibi alanlarda yaygın olarak kullanılmaktadır. Bu sistemler sayesinde kullanıcılar, bulundukları ortamdan bağımsız olarak cihazlara erişebilmekte, ortam koşullarını takip edebilmekte ve cihazların çalışma durumlarını uzaktan yönetebilmektedir.

Literatürde yer alan IoT tabanlı akıllı ev uygulamalarında genellikle mikrodenetleyici tabanlı sistemler tercih edilmektedir. Bu sistemlerde ESP8266 ve ESP32 gibi Wi-Fi haberleşme özelliğine sahip geliştirme kartları yaygın olarak kullanılmaktadır. ESP32; düşük maliyetli olması, Wi-Fi ve Bluetooth desteği sunması, yeterli sayıda giriş/çıkış pinine sahip olması ve farklı sensörler ile kolayca haberleşebilmesi nedeniyle IoT uygulamaları için uygun bir mikrodenetleyici platformu olarak öne çıkmaktadır. Bu nedenle akıllı priz, aydınlatma kontrolü ve ortam izleme uygulamalarında ESP32 tabanlı çözümler sıkça kullanılmaktadır.

IoT sistemlerinde cihazlar arası haberleşmenin hızlı ve güvenilir bir şekilde sağlanması önemli bir gerekliliktir. Bu amaçla literatürde yaygın olarak MQTT protokolünün kullanıldığı görülmektedir. MQTT, yayınla-abone ol mimarisiyle çalışan, düşük bant genişliği kullanan ve düşük gecikmeli veri iletimi sağlayan hafif bir haberleşme protokolüdür. Bu özellikleri sayesinde sensör verilerinin aktarılması, cihaz durumlarının izlenmesi ve uzaktan kontrol komutlarının iletilmesi için uygun bir yapı sunmaktadır. MQTT tabanlı sistemlerde cihazlar doğrudan birbirleriyle haberleşmek yerine bir broker üzerinden veri alışverişi yapmakta, bu durum sistemin daha düzenli ve genişletilebilir bir yapıda çalışmasına katkı sağlamaktadır.

Akıllı ev uygulamalarında merkezi kontrol yapısının oluşturulması için Raspberry Pi ve Node-RED gibi platformlardan da yararlanılmaktadır. Raspberry Pi, düşük güç tüketimli bir mini bilgisayar olması sayesinde yerel sunucu, MQTT broker ve veri işleme birimi olarak kullanılabilmektedir. Node-RED ise görsel programlama tabanlı yapısı sayesinde IoT cihazlarından gelen verilerin işlenmesini, yönlendirilmesini ve kullanıcı arayüzlerinin oluşturulmasını kolaylaştırmaktadır. Literatürde Node-RED tabanlı sistemlerin özellikle sensör verilerinin görselleştirilmesi, cihaz kontrol panellerinin hazırlanması ve farklı servislerle veri alışverişi yapılması amacıyla kullanıldığı görülmektedir.

IoT tabanlı ortam izleme çalışmalarında sıcaklık, nem ve ışık seviyesi gibi çevresel verilerin ölçülmesi de önemli bir yer tutmaktadır. Bu kapsamda DHT11 gibi sıcaklık ve nem sensörleri ile LDR tabanlı ışık algılama sensörleri yaygın olarak kullanılmaktadır. Elde edilen veriler yerel olarak ekranda gösterilebilmekte veya ThingSpeak gibi bulut tabanlı platformlara aktarılarak grafiksel olarak izlenebilmektedir. Böylece kullanıcılar ortam koşullarını anlık olarak takip edebilmekte ve sistemin geçmiş verilerini değerlendirebilmektedir.

Literatürdeki birçok akıllı priz ve aydınlatma kontrol sisteminde cihazların yalnızca mobil uygulama, web arayüzü veya tek bir kontrol yöntemi üzerinden yönetildiği görülmektedir. Bununla birlikte kullanıcı deneyimi açısından fiziksel buton, uzaktan kumanda, mobil kontrol paneli ve otomasyon arayüzlerinin aynı sistem içerisinde birlikte çalışması önemli bir avantaj sağlamaktadır. Farklı kontrol yöntemlerinin senkronize şekilde kullanılması, sistemin yalnızca

uzaktan erişime bağlı kalmamasını ve yerel kullanım senaryolarında da işlevsel olmasını sağlamaktadır.

Bu proje kapsamında geliştirilen sistem, literatürde yer alan IoT tabanlı akıllı ev uygulamalarından yararlanılarak tasarlanmıştır. Projede ESP32 mikrodenetleyicisi ile lamba ve fan kontrolü gerçekleştirilmiş, Raspberry Pi üzerinde çalışan MQTT broker ve Node-RED yapısı ile merkezi haberleşme altyapısı oluşturulmuştur. Lamba ve fan; fiziksel buton, IR uzaktan kumanda, MQTT tabanlı mobil uygulama ve Node-RED Dashboard üzerinden kontrol edilebilecek şekilde tasarlanmıştır. Ayrıca DHT11 sensörü ile sıcaklık ve nem verileri, LDR sensörü ile ortam aydınlık seviyesi ölçülmüş; elde edilen veriler OLED ekran ve ThingSpeak platformu üzerinden izlenebilir hâle getirilmiştir.

Bu yönüyle geliştirilen proje, yalnızca bir cihazın uzaktan açılıp kapatılmasını sağlayan basit bir kontrol sistemi olmaktan ziyade, farklı kontrol yöntemlerini, sensör verilerini, yerel görüntülemeyi ve bulut tabanlı veri takibini bir araya getiren bütünleşik bir IoT uygulaması niteliği taşımaktadır. Sistem, hem yerel hem de uzaktan kontrol imkânı sunması, çevresel verileri izleyebilmesi ve genişletilebilir bir mimariye sahip olması açısından literatürdeki benzer çalışmaları tamamlayıcı bir uygulama olarak değerlendirilebilir.

3.PROJEDE KULLANILAN YAZILIM VE DONANIM TEKNOLOJİLER

Bu proje kapsamında, IoT tabanlı akıllı priz ve ortam izleme sisteminin geliştirilmesi amacıyla çeşitli donanım bileşenleri, yazılım geliştirme ortamları ve haberleşme teknolojileri kullanılmıştır. Sistemde yer alan bu teknolojiler; cihazların kontrol edilmesi, sensör verilerinin okunması, verilerin kablosuz olarak aktarılması ve kullanıcı arayüzü üzerinden izlenmesi süreçlerinde önemli görevler üstlenmektedir. Projenin temel yapısı, mikrodenetleyici tabanlı kontrol birimi, sensörler, röle modülleri, yerel haberleşme altyapısı ve veri izleme platformlarından oluşmaktadır. Bu bileşenlerin birlikte çalışması sayesinde lamba ve fan gibi çıkış birimleri farklı kontrol yöntemleriyle yönetilebilmiş, ortam sıcaklığı, nem ve ışık seviyesi gibi veriler sistem üzerinden takip edilebilmiştir. Ayrıca yazılım tarafında kullanılan geliştirme ortamları ve haberleşme protokolleri, donanım birimlerinin birbiriyle uyumlu ve kararlı şekilde çalışmasını sağlamıştır. Aşağıda projede kullanılan yazılım ve donanım teknolojileri ayrı alt başlıklar hâlinde açıklanmaktadır.

3.1. Yazılım Teknolojileri

Projenin yazılım teknolojileri, sistemde kullanılan donanım bileşenlerinin kontrol edilmesi, sensörlerden alınan verilerin işlenmesi, cihaz durumlarının takip edilmesi ve farklı platformlar arasında veri alışverişinin sağlanması amacıyla kullanılmıştır. Bu kapsamda ESP32 mikrodenetleyicisinin programlanması, MQTT protokolü üzerinden haberleşmenin kurulması, Raspberry Pi üzerinde çalışan Mosquitto MQTT Broker ile veri iletiminin sağlanması, Node-RED ile akışların oluşturulması ve ThingSpeak platformuna veri aktarımı gerçekleştirilmiştir. Kullanılan yazılım araçları sayesinde sistem, hem yerel olarak hem de uzaktan kontrol edilebilen bütünleşik bir IoT uygulamasına dönüştürülmüştür.

3.1.1. Arduino IDE

Projenin yazılım geliştirme sürecinde Arduino IDE geliştirme ortamı kullanılmıştır. ESP32 mikrodenetleyicisi için gerekli olan kod yazımı, derleme ve karta yükleme işlemleri bu ortam üzerinden gerçekleştirilmiştir. Arduino IDE’nin sade arayüzü ve ESP32 desteği sayesinde mikrodenetleyici tabanlı uygulama geliştirme süreci daha kolay ve hızlı bir şekilde yürütülmüştür.

Arduino IDE üzerinde geliştirilen yazılım ile rölelerin kontrol edilmesi, DHT11 ve LDR sensörlerinden veri okunması, IR kumandadan gelen sinyallerin algılanması, buton durumunun kontrol edilmesi, OLED ekrana veri yazdırılması ve MQTT haberleşmesinin sağlanması işlemleri gerçekleştirilmiştir. Ayrıca Serial Monitor kullanılarak Wi-Fi bağlantı durumu, MQTT bağlantısı, sensör verileri ve sistemin çalışma aşamaları gözlemlenmiştir. Bu sayede yazılım geliştirme sırasında oluşabilecek bağlantı ve veri okuma hataları daha kolay takip edilmiştir.

Arduino IDE’nin geniş kütüphane desteği, projede farklı donanımların bir arada kullanılmasını kolaylaştırmıştır. Wi-Fi, MQTT, DHT11 sensörü, IR alıcı ve OLED ekran gibi bileşenler için hazır kütüphanelerden yararlanılmıştır. Bu durum, yazılım geliştirme sürecini hızlandırmış ve sistemin daha kararlı çalışmasına katkı sağlamıştır.

3.1.2. C/C++ Programlama Dili

ESP32 üzerinde çalışan kontrol yazılımı, Arduino geliştirme ortamı üzerinde C/C++ programlama dili kullanılarak geliştirilmiştir. C/C++ dili, mikrodenetleyici tabanlı sistemlerde donanıma yakın çalışma imkânı sunduğu için giriş-çıkış işlemlerinin hızlı ve verimli bir şekilde gerçekleştirilmesini sağlamaktadır. Bu nedenle projede sensör okuma, röle kontrolü, haberleşme ve ekran işlemleri C/C++ tabanlı kod yapısı ile gerçekleştirilmiştir.

Sistemde lamba ve fan rölelerinin kontrol edilmesi, buton durumunun okunması, IR kumandadan gelen sinyallerin çözümlenmesi, DHT11 sıcaklık-nem sensöründen ve LDR ışık sensöründen alınan verilerin işlenmesi bu yazılım dili aracılığıyla sağlanmıştır. Ayrıca OLED ekran üzerinde sistem bilgilerinin gösterilmesi ve MQTT üzerinden veri gönderme-alma işlemleri de C/C++ kod yapısı içerisinde oluşturulmuştur.

C/C++ dilinin tercih edilmesi, sistemin gerçek zamanlı çalışmasına katkı sağlamıştır. Kullanıcının butona basması, kumandadan komut göndermesi veya Node-RED Dashboard üzerinden kontrol işlemi yapması durumunda ESP32 bu komutları hızlı bir şekilde algılayarak ilgili çıkış birimini kontrol etmektedir. Böylece sistemin kararlı, hızlı tepki veren ve kontrol edilebilir bir yapıda çalışması sağlanmıştır.

3.1.3. MQTT Protokolü

Projede ESP32, Raspberry Pi, Node-RED Dashboard ve IoT MQTT Panel mobil uygulaması arasındaki haberleşmenin sağlanması amacıyla MQTT protokolü kullanılmıştır. MQTT, yayınla-abone ol mimarisiyle çalışan hafif bir haberleşme protokolüdür. Bu mimaride cihazlar doğrudan birbirine veri göndermek yerine belirlenen konu başlıkları üzerinden MQTT broker aracılığıyla haberleşmektedir.

Sistemde lamba ve fan kontrol komutları belirlenen MQTT topic yapıları üzerinden ESP32'ye iletilmiştir. ESP32, ilgili topic'lere abone olarak gelen açma-kapama komutlarını almış ve bu komutlara göre röle çıkışlarını kontrol etmiştir. Aynı zamanda sıcaklık, nem, ışık seviyesi, lamba durumu ve fan durumu gibi bilgiler ESP32 tarafından MQTT üzerinden yayınlanmıştır. Bu sayede sistem durumları Node-RED ve diğer izleme platformları tarafından takip edilebilir hâle getirilmiştir.

MQTT protokolünün düşük bant genişliği kullanması ve hızlı veri iletimi sağlaması, IoT uygulamaları için önemli bir avantaj oluşturmaktadır. Bu proje kapsamında MQTT sayesinde ESP32 ile Raspberry Pi arasında düşük gecikmeli ve düzenli bir veri alışverişi sağlanmıştır. Böylece sistem hem yerel ağ üzerinden kontrol edilebilir hâle gelmiş hem de farklı arayüzlerle kolayca entegre edilebilmiştir.

3.1.4. Mosquitto MQTT Broker

Projede MQTT haberleşmesini sağlamak amacıyla Raspberry Pi üzerinde Mosquitto MQTT Broker kullanılmıştır. Mosquitto, ESP32, Node-RED ve IoT MQTT Panel mobil uygulaması arasında merkezi haberleşme noktası olarak görev yapmıştır. Bu yapı sayesinde sistemde kullanılan cihazlar tek bir broker üzerinden veri alışverişi yapabilmiştir.

ESP32, Mosquitto broker üzerinde belirlenen topic'lere abone olarak lamba ve fan için gelen kontrol komutlarını almıştır. Aynı zamanda sensörlerden okunan sıcaklık, nem ve ışık

seviyesi gibi veriler ile lamba ve fan durum bilgileri de yine MQTT broker üzerinden yayınlanmıştır. Node-RED ise bu topic'leri dinleyerek gelen verileri işlemiş ve kullanıcı arayüzünde gösterilecek hâle getirmiştir.

Mosquitto MQTT Broker'ın Raspberry Pi üzerinde yerel olarak çalışması, sistemin daha kararlı ve bağımsız bir haberleşme altyapısına sahip olmasını sağlamıştır. Genel kullanıma açık broker yapıları yerine yerel broker kullanılması, verilerin yerel ağ içerisinde kalmasına ve sistemin internet bağlantısından daha az etkilenmesine katkı sağlamıştır. Bu nedenle Mosquitto, projenin merkezi haberleşme altyapısında önemli bir yazılım bileşeni olarak kullanılmıştır.

3.1.5. Node-RED

Projede sistemin görsel olarak yönetilebilmesi, MQTT üzerinden gelen verilerin işlenebilmesi ve kontrol akışlarının oluşturulabilmesi amacıyla Node-RED kullanılmıştır. Node-RED, sürükle-bırak mantığıyla çalışan görsel programlama tabanlı bir platformdur. Bu yapı sayesinde karmaşık kod yapıları yazmadan, farklı giriş ve çıkış bileşenleri arasında veri akışı oluşturulabilmektedir.

Raspberry Pi üzerinde çalışan Node-RED aracılığıyla MQTT broker'dan gelen sıcaklık, nem, ışık seviyesi, lamba durumu ve fan durumu gibi veriler alınmıştır. Bu veriler Node-RED akışları içerisinde işlenmiş, gerekli yönlendirmeler yapılmış ve kullanıcı arayüzünde gösterilmek üzere Dashboard bileşenlerine aktarılmıştır. Ayrıca Node-RED üzerinden oluşturulan kontrol butonları sayesinde lamba ve fan için açma-kapama komutları MQTT topic'leri üzerinden ESP32'ye gönderilmiştir.

Node-RED'in projede kullanılması, sistemin test edilmesini ve geliştirilmesini kolaylaştırmıştır. Akış yapısı üzerinden hangi verinin nereden geldiği, hangi işlemden geçtiği ve hangi çıkışa yönlendirildiği kolayca takip edilebilmiştir. Bu sayede hem kontrol yapısı hem de veri izleme yapısı daha anlaşılır ve düzenli hâle getirilmiştir.

3.1.6. Node-RED Dashboard

Kullanıcının sistemi web tabanlı bir arayüz üzerinden kontrol edebilmesi ve sistem durumlarını takip edebilmesi için Node-RED Dashboard kullanılmıştır. Node-RED Dashboard, Node-RED üzerinde oluşturulan akışlara görsel arayüz bileşenleri eklenmesini sağlayan bir yapıdır. Bu sayede kullanıcı, tarayıcı üzerinden sisteme erişerek lamba ve fan kontrolü yapabilmiş, sensör verilerini anlık olarak görüntüleyebilmiştir.

Dashboard üzerinde lamba ve fan için kontrol anahtarları oluşturulmuş, bu anahtarlar MQTT topic'leri ile ilişkilendirilmiştir. Kullanıcı Dashboard üzerinden bir komut gönderdiğinde, Node-RED bu komutu MQTT broker'a iletmiş ve ESP32 ilgili komutu alarak röle çıkışını kontrol etmiştir. Aynı zamanda ESP32'den gelen cihaz durum bilgileri de Dashboard üzerinde gösterilmiştir. Böylece kullanıcı, lamba ve fanın açık ya da kapalı olduğunu arayüz üzerinden takip edebilmiştir.

Sıcaklık, nem ve ışık seviyesi gibi sensör verileri de Node-RED Dashboard üzerinden izlenebilir hâle getirilmiştir. Bu yapı, sistemin yalnızca fiziksel olarak değil, aynı zamanda web tabanlı bir kontrol paneli üzerinden de yönetilebilmesini sağlamıştır. Böylece proje,

klasik yerel kontrol sistemlerinden farklı olarak IoT tabanlı uzaktan izleme ve kontrol özelliği kazanmıştır.

3.1.7. JSON Veri Yapısı

Projede Node-RED akışlarının düzenlenmesi, dışa aktarılması ve tekrar içe aktarılması için JSON veri yapısından yararlanılmıştır. Node-RED üzerinde oluşturulan akışlar JSON formatında saklanabilmekte ve gerektiğinde başka bir sisteme aktarılabilmektedir. Bu özellik, proje akışlarının yedeklenmesi ve yeniden kurulması açısından önemli bir kolaylık sağlamıştır.

JSON yapısı, verilerin düzenli ve okunabilir bir biçimde temsil edilmesini sağlar. Node-RED üzerinde kullanılan düğümler, bağlantılar, MQTT topic bilgileri ve Dashboard bileşenleri JSON formatında ifade edilebilmektedir. Bu sayede oluşturulan akışlar gerektiğinde düzenlenebilir, paylaşılabilir veya farklı bir Raspberry Pi üzerinde tekrar kullanılabilir.

Bu projede JSON veri yapısı özellikle Node-RED akışlarının korunması ve sistemin tekrar kurulabilir hâle getirilmesi açısından kullanılmıştır. Proje üzerinde yapılan değişikliklerin JSON formatında dışa aktarılabilmesi, hata durumlarında veya sistem yeniden kurulmak istendiğinde önemli bir avantaj sağlamıştır. Böylece yazılım tarafında daha düzenli ve sürdürülebilir bir yapı elde edilmiştir.

3.1.8. IoT MQTT Panel Mobil Uygulaması

Sistemin mobil cihaz üzerinden kontrol edilebilmesi için IoT MQTT Panel mobil uygulaması kullanılmıştır. Bu uygulama, MQTT protokolü ile çalışan cihazların telefondan kontrol edilebilmesini sağlayan bir arayüz sunmaktadır. Kullanıcı, uygulama üzerinde oluşturulan buton veya anahtar bileşenleri aracılığıyla belirlenen MQTT topic'lerine komut gönderebilmektedir.

Bu projede IoT MQTT Panel uygulaması ile lamba ve fan için açma-kapama komutları oluşturulmuştur. Kullanıcı mobil uygulama üzerinden komut gönderdiğinde, bu komut MQTT broker'a iletilmiş ve ESP32 tarafından alınarak ilgili röle çıkışı kontrol edilmiştir. Böylece sistem, yalnızca Node-RED Dashboard üzerinden değil, aynı zamanda mobil cihaz üzerinden de yönetilebilir hâle getirilmiştir.

IoT MQTT Panel uygulamasının kullanılması, projeye mobil kontrol özelliği kazandırmıştır. Bu sayede kullanıcı, aynı ağ içerisinde mobil cihazını kullanarak lamba ve fanı kontrol edebilmiş, sistemin uzaktan yönetilebilirliğini test edebilmiştir. Uygulamanın MQTT topic yapısı ile çalışması, Node-RED ve ESP32 ile kolay entegrasyon sağlamıştır.

3.1.9. ThingSpeak

Projede sıcaklık, nem, ışık seviyesi, lamba durumu ve fan durumu gibi verilerin internet üzerinden izlenebilmesi amacıyla ThingSpeak platformu kullanılmıştır. ThingSpeak, IoT projelerinde sensör verilerinin kaydedilmesi, saklanması ve grafiksel olarak görüntülenmesi için kullanılan bulut tabanlı bir veri izleme platformudur.

Sistemden elde edilen veriler Node-RED üzerinden işlenmiş ve ThingSpeak platformuna aktarılmıştır. Böylece sıcaklık, nem ve ışık seviyesi gibi çevresel verilerin zaman içerisindeki

değişimi grafikler üzerinden takip edilebilmiştir. Ayrıca lamba ve fan durum bilgilerinin de platforma aktarılmasıyla cihazların açık veya kapalı olma durumları izlenebilir hâle getirilmiştir.

ThingSpeak kullanımı, projenin veri izleme yönünü güçlendirmiştir. Kullanıcı yalnızca anlık değerleri değil, belirli zaman aralıklarındaki değişimleri de gözlemleyebilmiştir. Bu özellik, IoT sistemlerinde veri takibi ve analiz açısından önemli bir avantaj sağlamaktadır. Projede ThingSpeak'in kullanılması, sistemin yalnızca kontrol odaklı değil, aynı zamanda veri izleme ve kayıt tutma özelliğine sahip bir yapı kazanmasını sağlamıştır.

3.2.Donanım Teknolojileri

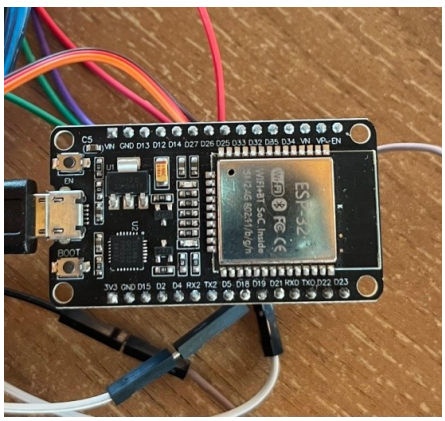
Projenin donanım teknolojileri, sistemin fiziksel olarak çalışmasını sağlayan kontrol, algılama, haberleşme ve çıkış birimlerinden oluşmaktadır. Bu kapsamda ESP32 mikrodenetleyicisi sistemin ana kontrol birimi olarak kullanılmış; sensörler aracılığıyla ortam verileri alınmış, röle modülleri ile lamba ve fan gibi çıkış birimleri kontrol edilmiştir. Ayrıca OLED ekran, IR alıcı modülü, buton ve Raspberry Pi gibi bileşenler sisteme yerel kontrol, görüntüleme ve merkezi haberleşme özellikleri kazandırmıştır. Aşağıda projede kullanılan donanım teknolojileri ve bu teknolojilerin sistem içerisindeki görevleri açıklanmaktadır.

3.2.1. ESP32 Mikrodenetleyici

Sistemin merkezî kontrol birimi olarak ESP32 mikrodenetleyicisi kullanılmıştır. ESP32, dahili Wi-Fi özelliği sayesinde internet üzerinden haberleşme gerçekleştirebilmekte ve IoT uygulamaları için güçlü bir platform sunmaktadır. Projede ESP32, kullanıcıdan gelen komutları işleyen, farklı kontrol birimlerinden gelen verileri analiz eden ve röle modülü aracılığıyla yük kontrolünü gerçekleştiren "ana işlemci birimi" olarak görev yapmaktadır.

Sistem mimarisinde ESP32; mobil uygulama üzerinden MQTT protokolü ile gönderilen komutları almakta, IR alıcı modülünden gelen sinyalleri çözümlemekte ve fiziksel buton girişlerini sürekli izleyerek kullanıcı etkileşimini sağlamaktadır. Bu girişlerden gelen veriler doğrultusunda röle modülünü kontrol ederek lambanın açılıp kapatılmasını sağlamaktadır.

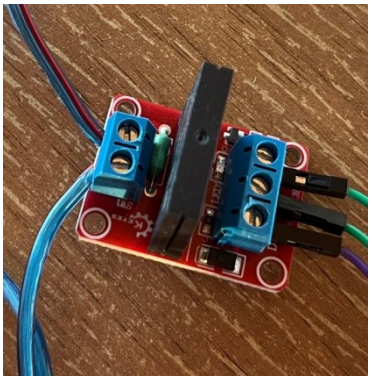
ESP32'nin tercih edilme nedenleri arasında dahili Wi-Fi desteği, yüksek işlem kapasitesi, geniş GPIO pin yapısı ve IoT uygulamalarına uygun mimarisi bulunmaktadır. Ayrıca düşük maliyetli olması ve geniş geliştirici desteği sayesinde proje için uygun bir kontrol birimi olarak değerlendirilmiştir.



Şekil 3.2.1. ESP32

3.2.2 Röle Modülü

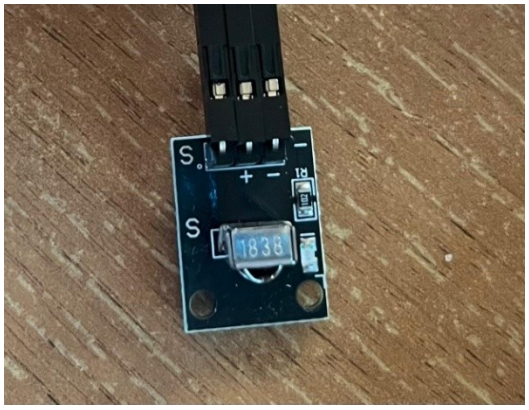
Sistemde elektriksel yükün kontrol edilmesi amacıyla röle modülü kullanılmıştır. Röle, düşük gerilimli kontrol sinyalleri ile yüksek gerilimli bir devrenin açılıp kapatılmasını sağlayan bir anahtarlama elemanıdır.Sistem mimarisinde röle modülü, “yük kontrol birimi” olarak görev yapmaktadır. ESP32’den gelen dijital sinyaller doğrultusunda röle aktif veya pasif hâle getirilmekte ve buna bağlı olarak lambanın enerjisi kontrol edilmektedir. Kullanıcı tarafından verilen komutlar (buton, kumanda veya mobil uygulama aracılığıyla) ESP32 tarafından işlendikten sonra röleye iletilmekte ve fiziksel yük üzerinde doğrudan etki oluşturmaktadır.Rölenin tercih edilme nedenleri arasında yüksek gerilimli yükleri güvenli şekilde kontrol edebilmesi, basit bağlantı yapısı ve mikrodenetleyiciler ile kolay entegrasyon sağlaması bulunmaktadır.



Şekil 3.2.3. Röle modülü

3.2.3. IR Alıcı Modülü

Projede kablosuz kontrol yöntemlerinden biri olarak IR (kızılötesi) alıcı modülü kullanılmıştır. Bu modül, uzaktan kumanda tarafından gönderilen kızılötesi sinyalleri algılayarak ESP32’ye iletmektedir.Sistem mimarisinde IR alıcı modülü, “kablosuz veri alma birimi” olarak görev yapmaktadır. Kullanıcı kumanda üzerinden bir tuşa bastığında oluşan sinyal modül tarafından algılanmakta ve ESP32 tarafından çözümlenerek ilgili komut çalıştırılmaktadır. Bu sayede sistem, internet bağlantısına ihtiyaç duymadan da uzaktan kontrol edilebilmektedir.IR alıcı modülünün tercih edilme nedenleri arasında düşük maliyet, kolay kullanım ve uzaktan kontrol imkânı sağlaması yer almaktadır.



Şekil 3.2.3. IR Alıcı Modülü

3.2.4. IR Uzaktan Kumanda

IR alıcı modülü ile birlikte çalışan uzaktan kumanda, kullanıcıya pratik bir kablosuz kontrol imkânı sunmaktadır. Kumanda üzerinden gönderilen sinyaller IR alıcı modül tarafından algılanarak ESP32'ye iletilmektedir.Sistem içerisinde kumanda, "kullanıcı kontrol arayüzü" olarak görev yapmaktadır. Belirli tuşlara karşılık gelen kodlar yazılım tarafında tanımlanmış olup, kullanıcı bu tuşları kullanarak lambayı açıp kapatabilmektedir. Bu yöntem, özellikle yerel kullanım senaryolarında hızlı ve kolay bir kontrol sağlamaktadır.



Şekil 3.2.4 IR Uzaktan Kumanda

3.2.5. Buton

Sistemde yerel kullanıcı etkileşimini sağlamak amacıyla push button (basmalı anahtar) kullanılmıştır. Bu buton, kullanıcının doğrudan cihaz üzerinden kontrol işlemi gerçekleştirmesine olanak tanımaktadır.Sistem mimarisinde buton, "yerel kontrol birimi" olarak görev yapmaktadır. Butona basıldığında ESP32 giriş pininde lojik seviye değişimi algılanmakta ve buna bağlı olarak lamba durumu değiştirilmektedir. Buton kontrolü yazılım tarafında debounce mekanizması ile desteklenerek hatalı tetiklemelerin önüne geçilmiştir.Butonun tercih edilme nedenleri arasında basit yapısı, düşük maliyeti ve kullanıcıya doğrudan kontrol imkânı sunması bulunmaktadır.

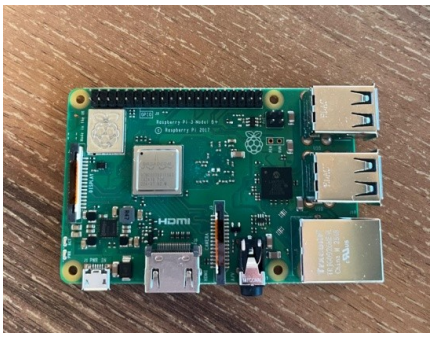
.



Şekil 3.2.5. Buton

3.2.7. Raspberry Pi

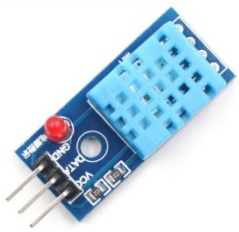
Projenin ilerleyen aşamalarında sistemin merkezi kontrol ve haberleşme birimi olarak Raspberry Pi kullanılması planlanmaktadır. Raspberry Pi, küçük boyutlu bir tek kart bilgisayar olup, üzerinde çalıştırılacak yazılımlar sayesinde IoT sistemlerinde sunucu görevi üstlenebilmektedir.Sistem mimarisinde Raspberry Pi, “merkezi yönetim ve haberleşme birimi” olarak görev yapacaktır. MQTT protokolü için Mosquitto broker’ın Raspberry Pi üzerinde kurulması ile birlikte, sistemde kullanılan mevcut public broker yapısının yerini yerel ve daha güvenli bir haberleşme altyapısı alacaktır. Bu sayede ESP32 ile mobil cihaz arasındaki veri iletişimi doğrudan Raspberry Pi üzerinden gerçekleştirilecektir.Ayrıca Raspberry Pi üzerinde Node-RED platformunun çalıştırılması planlanmaktadır. Bu yapı sayesinde kullanıcılar, web tabanlı bir arayüz üzerinden sisteme erişebilecek, cihazları kontrol edebilecek ve sistem durumunu izleyebilecektir. Böylece mobil uygulamaya ek olarak daha kapsamlı ve kullanıcı dostu bir kontrol paneli oluşturulacaktır.Raspberry Pi’nin tercih edilme nedenleri arasında düşük maliyetli olması, Linux tabanlı işletim sistemi desteği, MQTT ve Node-RED gibi IoT teknolojileri ile kolay entegrasyon sağlaması ve birden fazla cihazın merkezi olarak yönetilebilmesine imkân tanıması yer almaktadır. Bu özellikleri sayesinde sistem, daha güvenli, esnek ve genişletilebilir bir IoT mimarisine dönüştürülmektedir.



Şekil 3.2.7. Rasperry Pi

3.2.8.DHT11 Sıcaklık ve Nem Sensörü

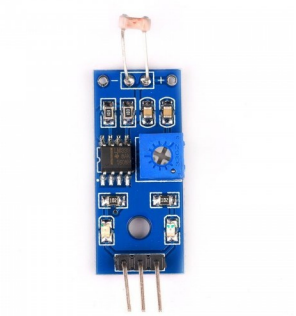
Projede ortam sıcaklığı ve nem değerlerinin ölçülmesi amacıyla DHT11 sıcaklık ve nem sensörü kullanılmıştır. Bu sensör, ortamdan aldığı sıcaklık ve bağıl nem bilgilerini dijital sinyal olarak ESP32 mikrodenetleyicisine iletmektedir. Sistem içerisinde DHT11 sensörü, “ortam izleme birimi” olarak görev yapmaktadır.ESP32 tarafından okunan sıcaklık ve nem verileri yazılım içerisinde işlenerek hem OLED ekran üzerinde kullanıcıya gösterilmekte hem de MQTT haberleşmesi aracılığıyla Node-RED ve ThingSpeak platformlarına aktarılmaktadır. Böylece kullanıcı, ortam koşullarını anlık olarak takip edebilmekte ve sıcaklık ile nem değerlerindeki değişimleri sistem üzerinden gözlemleyebilmektedir.



Şekil 3.2.8 DHT11

3.2.9. LDR Sensör Modülü

Projede ortamın aydınlık seviyesini algılamak amacıyla LDR sensör modülü kullanılmıştır. Sensörden alınan analog değerler ESP32 tarafından okunarak ortamın aydınlık veya karanlık durumu belirlenmiştir. Elde edilen ışık bilgisi OLED ekranda gösterilmiş ve MQTT üzerinden Node-RED ile ThingSpeak platformlarına aktarılmıştır. LDR sensör modülü, düşük maliyetli olması ve ortam ışığını kolay bir şekilde algılayabilmesi nedeniyle projede tercih edilmişti



Şekil 3.2.9. LDR Modül

3.2.10.5V DC Fan

Projede ortam havalandırmasını temsil eden çıkış birimi olarak DC fan kullanılmıştır. Fanın açma ve kapama işlemleri ESP32 tarafından kontrol edilen röle modülü aracılığıyla gerçekleştirilmiştir. Sistem içerisinde fan; IR kumanda, Node-RED Dashboard ve MQTT tabanlı mobil uygulama üzerinden kontrol edilebilecek şekilde tasarlanmıştır. Ayrıca fan durumu OLED ekranda gösterilmiş ve MQTT üzerinden Node-RED ile ThingSpeak platformlarına aktarılmıştır.



Şekil 3.2.10 5V DC Fan

3.2.11. OLED Ekran

Projede ortam havalandırmasını temsil eden çıkış birimi olarak 5V DC fan kullanılmıştır. Fan, sistemde kontrol edilen elektriksel yüklerden biri olup ESP32 mikrodenetleyicisi tarafından

doğrudan değil, röle modülü aracılığıyla kontrol edilmektedir. Bu sayede ESP32’nin düşük gerilimli kontrol sinyali ile fanın besleme hattı anahtarlanmış ve fanın güvenli şekilde açılıp kapatılması sağlanmıştır.

Sistem içerisinde fan; IR uzaktan kumanda, Node-RED Dashboard ve MQTT tabanlı mobil uygulama üzerinden kontrol edilebilecek şekilde tasarlanmıştır. Kullanıcı tarafından gönderilen açma veya kapama komutları MQTT haberleşmesi üzerinden ESP32’ye iletilmekte, ESP32 ise ilgili röle çıkışını aktif veya pasif duruma getirerek fanın çalışma durumunu değiştirmektedir. Böylece fan hem yerel hem de uzaktan kontrol edilebilen bir çıkış birimi olarak sisteme entegre edilmiştir.

Fan durum bilgisi OLED ekran üzerinde kullanıcıya gösterilmiş, aynı zamanda MQTT üzerinden Node-RED ve ThingSpeak platformlarına aktarılmıştır. Bu sayede fanın açık veya kapalı olma durumu hem cihaz üzerinden hem de internet tabanlı izleme arayüzleri üzerinden takip edilebilmiştir. Fanın projede kullanılması, sistemin yalnızca aydınlatma kontrolüyle sınırlı kalmamasını sağlamış ve IoT tabanlı farklı elektriksel yüklerin kontrol edilebileceğini göstermiştir.



Şekil 3.2.11. OLED Ekran

4.NESNELERİN İNTERNETİ (IoT) NEDİR?

Nesnelerin İnterneti, İngilizce adıyla Internet of Things (IoT), fiziksel nesnelerin internet veya yerel ağ üzerinden birbirleriyle, kullanıcılarla ve farklı dijital platformlarla haberleşebilmesini sağlayan teknoloji yapısıdır. Bu yapı sayesinde günlük yaşamda kullanılan cihazlar yalnızca manuel olarak kontrol edilen araçlar olmaktan çıkarak veri toplayabilen, veri aktarabilen, izlenebilen ve uzaktan yönetilebilen akıllı sistemlere dönüşmektedir.

IoT teknolojisinin temelinde sensörler, mikrodenetleyiciler, haberleşme protokolleri, veri işleme platformları ve kullanıcı arayüzleri yer almaktadır. Sensörler ortamdan sıcaklık, nem, ışık, hareket, basınç veya konum gibi verileri toplarken; kontrol birimleri bu verileri işler ve gerekli durumlarda çıkış birimlerini yönetir. Toplanan veriler Wi-Fi, Bluetooth, ZigBee, LoRa, GSM veya Ethernet gibi haberleşme yöntemleriyle başka cihazlara, yerel sunuculara ya da bulut platformlarına aktarılabilir. Kullanıcılar ise mobil uygulamalar, web arayüzleri veya kontrol panelleri üzerinden bu sistemleri takip edebilir ve yönetebilir.

Günlük hayatta IoT teknolojileri birçok alanda kullanılmaktadır. Akıllı ev sistemlerinde lambaların, prizlerin, klimaların, güvenlik kameralarının ve kapı kilitlerinin uzaktan kontrol edilmesi IoT uygulamalarına örnek olarak gösterilebilir. Bunun yanında akıllı saatler, sağlık takip cihazları, akıllı sayaçlar, araç takip sistemleri, endüstriyel üretim hatları, tarımsal sulama sistemleri ve çevresel izleme sistemleri de IoT teknolojilerinin yaygın kullanım alanları arasında yer almaktadır.

IoT sistemleri, kullanıcılara zaman ve mekândan bağımsız kontrol imkânı sunması açısından önemli avantajlar sağlamaktadır. Kullanıcılar, bulundukları ortamdan bağımsız olarak cihazların durumunu izleyebilir, gerektiğinde cihazlara komut gönderebilir ve sistemin çalışma koşullarını takip edebilir. Bu durum hem kullanıcı konforunu artırmakta hem de enerji yönetimi, güvenlik ve verimlilik açısından önemli katkılar sağlamaktadır. Örneğin bir aydınlatma sistemi yalnızca ihtiyaç duyulduğunda çalıştırılarak enerji tasarrufu sağlanabilir veya bir ortamın sıcaklık ve nem değerleri uzaktan takip edilerek gerekli önlemler alınabilir.

Nesnelerin İnterneti teknolojisi yalnızca bireysel kullanımda değil, endüstriyel ve kurumsal alanlarda da önemli bir yere sahiptir. Endüstriyel otomasyon sistemlerinde makinelerin çalışma durumlarının izlenmesi, üretim verilerinin kaydedilmesi ve arızaların önceden tespit edilmesi IoT sistemleri ile daha etkili hâle getirilebilmektedir. Tarım alanında toprak nemi, hava sıcaklığı ve sulama durumu takip edilerek daha verimli üretim yapılabilir. Sağlık alanında ise hastaların bazı yaşamsal verileri uzaktan izlenerek erken müdahale imkânı sağlanabilir.

Sonuç olarak IoT, fiziksel cihazların dijital sistemlerle bütünleşmesini sağlayan önemli bir teknolojidir. Bu teknoloji sayesinde cihazlar daha akıllı, izlenebilir, kontrol edilebilir ve verimli hâle gelmektedir. Günümüzde akıllı evlerden endüstriyel otomasyona, sağlıktan tarıma kadar birçok alanda kullanılan IoT sistemleri, gelecekte daha fazla cihazın birbirine bağlı çalışmasına ve daha gelişmiş otomasyon yapılarının ortaya çıkmasına katkı sağlayacaktır.

5. PROJENİN GERÇEKLENMESİ

5.1. Giriş

Bu bölümde, IoT tabanlı akıllı priz ve ortam izleme sisteminin uygulama aşamaları açıklanmaktadır. Projenin gerçeklenmesi sürecinde öncelikle sistemde kullanılacak donanım bileşenleri belirlenmiş ve bu bileşenlerin ESP32 mikrodenetleyicisi ile bağlantıları yapılmıştır. Ardından lamba ve fan kontrolü, sensör verilerinin okunması, OLED ekranda sistem bilgilerinin gösterilmesi, IR kumanda ve buton ile yerel kontrol işlemleri gerçekleştirilmiştir. Ayrıca Raspberry Pi üzerinde MQTT broker ve Node-RED yapısı kullanılarak sistemin haberleşme ve kontrol altyapısı oluşturulmuştur. Bu bölümde donanım tasarımı, yazılım tasarımı ve sistemin çalışma yapısı sırasıyla ele alınmaktadır.

5.2 Donanım Tasarımı

Bu bölümde, IoT tabanlı akıllı priz ve ortam izleme sisteminin donanım tasarım süreci açıklanmaktadır. Donanım tasarımı, ESP32 mikrodenetleyicisi merkez alınarak oluşturulmuş ve sistemde kullanılan sensörler, röle modülleri, kullanıcı kontrol birimleri ve görüntüleme elemanları bu yapı etrafında konumlandırılmıştır. Sistemde DHT11 ve LDR sensörleri ortam verilerinin alınmasını sağlarken, IR alıcı modülü ve buton kullanıcıdan gelen kontrol komutlarının algılanmasında kullanılmıştır. Lamba ve fan gibi çıkış birimleri röle modülleri üzerinden kontrol edilmiş, OLED ekran ile sistemin anlık durum bilgileri kullanıcıya gösterilmiştir. Ayrıca Raspberry Pi, MQTT broker ve Node-RED yapısı için merkezi haberleşme birimi olarak donanım mimarisine dâhil edilmiştir.

5.2.1. Sistem Donanım Mimarisi

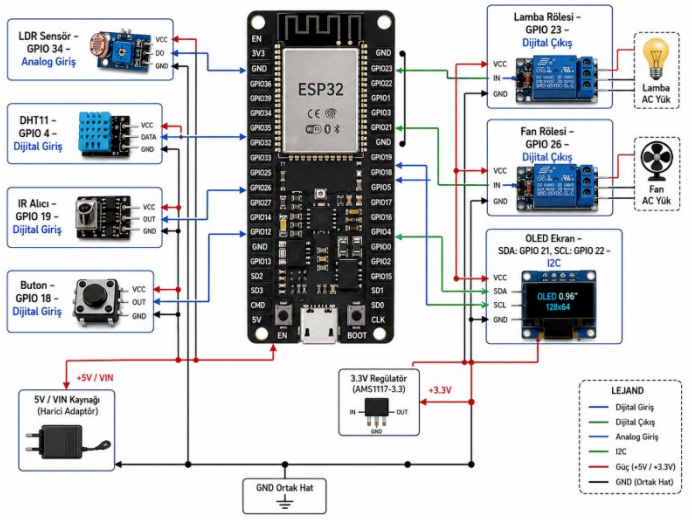
Projenin donanım mimarisi, ESP32 mikrodenetleyicisi merkez alınarak tasarlanmıştır. Sistem içerisinde ESP32, sensörlerden gelen verileri okuyan, kullanıcıdan gelen kontrol komutlarını işleyen ve çıkış birimlerini yöneten ana kontrol birimi olarak görev yapmaktadır. DHT11 sıcaklık ve nem sensörü ile ortam sıcaklığı ve nem bilgisi alınırken, LDR sensör modülü ile ortamın ışık seviyesi algılanmaktadır. IR alıcı modülü, uzaktan kumandadan gelen kızılötesi sinyalleri ESP32'ye iletmekte; buton ise kullanıcıya doğrudan yerel kontrol imkânı sağlamaktadır.

Sistemde lamba ve fan, ESP32 tarafından kontrol edilen röle modülleri aracılığıyla açılıp kapatılmaktadır. Böylece düşük gerilimli mikrodenetleyici çıkışları ile elektriksel yüklerin güvenli şekilde kontrol edilmesi sağlanmıştır. OLED ekran, sistemin anlık durum bilgilerinin kullanıcıya yerel olarak gösterilmesi amacıyla kullanılmıştır. Ekran üzerinde lamba ve fan durumları ile sıcaklık, nem ve ışık bilgileri görüntülenmektedir.

Raspberry Pi ise sistemde haberleşme ve merkezi kontrol birimi olarak görev yapmaktadır. Raspberry Pi üzerinde çalışan Mosquitto MQTT broker ve Node-RED yapısı sayesinde ESP32 ile kullanıcı arayüzleri arasında veri alışverişi sağlanmıştır. ESP32, Wi-Fi bağlantısı üzerinden Raspberry Pi ile haberleşmekte; sensör verileri, cihaz durumları ve kontrol komutları MQTT protokolü aracılığıyla iletilmektedir. Bu yapı sayesinde sistem hem yerel donanım elemanlarıyla hem de uzaktan kontrol arayüzleriyle birlikte çalışan bütünleşik bir IoT mimarisine dönüştürülmüştür.

5.2.2. ESP32 Pin Bağlantı Planı

Sistemde kullanılan donanım bileşenlerinin ESP32 mikrodenetleyicisi ile düzenli ve kararlı çalışabilmesi için pin bağlantı planı oluşturulmuştur. Bu plan kapsamında lamba rölesi GPIO 23 pinine, fan rölesi GPIO 26 pinine, buton GPIO 18 pinine ve IR alıcı modülü GPIO 19 pinine bağlanmıştır. DHT11 sıcaklık ve nem sensörü GPIO 4 pini üzerinden veri iletirken, LDR sensör modülü ortam ışık seviyesinin okunması için GPIO 34 analog giriş pinine bağlanmıştır. OLED ekran ise I2C haberleşme protokolü ile kullanılmış olup SDA hattı GPIO 21, SCL hattı ise GPIO 22 pinine bağlanmıştır. Bu bağlantı yapısı sayesinde ESP32; sensör verilerini okuyabilmekte, kullanıcıdan gelen kontrol komutlarını algılayabilmekte ve röle modülleri üzerinden lamba ile fanı kontrol edebilmektedir.



Şekil 5.2.2. ESP32 Pin Bağlantıları

5.2.3. Pertinaks Üzerindeki Donanım Yerleşimi

Projenin donanım bağlantılarının daha düzenli, sağlam ve taşınabilir bir yapıya sahip olması amacıyla devre elemanları pertinaks üzerine yerleştirilmiştir. Pertinaks kullanımı sayesinde breadboard üzerinde yapılan geçici bağlantılar daha kalıcı hâle getirilmiş, jumper kabloların gevşemesi ve bağlantı kopması gibi sorunların önüne geçilmesi amaçlanmıştır. Bu yapı, sistemin proje kutusu içerisine daha düzenli şekilde yerleştirilmesine de olanak sağlamıştır.

Pertinaks üzerindeki yerleşimde ESP32 mikrodenetleyicisi sistemin merkezi kontrol birimi olarak konumlandırılmıştır. ESP32'ye bağlı olan sensörler, kullanıcı kontrol birimleri ve röle modülleri, bağlantı karmaşasını azaltacak şekilde farklı bölgelerde yerleştirilmiştir. Bu tasarımda DHT11 sıcaklık ve nem sensörü ile LDR ışık sensörü giriş birimleri olarak kullanılmış; bu sensörlerden alınan veriler ESP32 tarafından okunarak yazılım içerisinde işlenmiştir. IR alıcı modülü ve buton ise kullanıcıdan gelen kontrol komutlarının algılanması için sisteme dâhil edilmiştir.

Röle modülleri, lamba ve fan gibi çıkış birimlerinin kontrol edilmesi amacıyla pertinaks üzerinde ESP32'ye uygun mesafede konumlandırılmıştır. Rölelerin ESP32'den gelen düşük gerilimli kontrol sinyallerini kullanarak yükleri anahtarlaması sağlanmıştır. Bu sayede mikrodenetleyici devresi ile lamba ve fan gibi çıkış birimleri arasında kontrollü bir anahtarlama yapısı oluşturulmuştur. Röle bağlantılarında güç hattı ile sinyal hattının mümkün

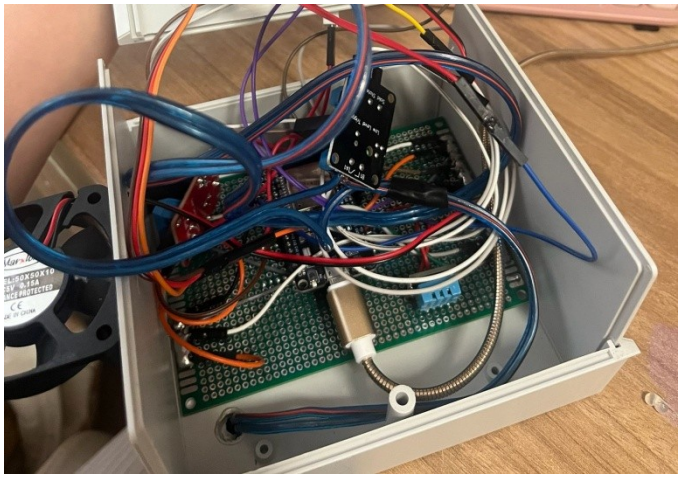
olduğunca düzenli ayrılmasına dikkat edilmiştir. Böylece hem bağlantı takibi kolaylaştırılmış hem de olası kısa devre risklerinin azaltılması hedeflenmiştir.

OLED ekran bağlantısı, ESP32'nin I2C haberleşme hattı üzerinden gerçekleştirilmiştir. OLED ekranın sistemdeki görevi; lamba ve fan durumları ile sıcaklık, nem ve ışık seviyesi gibi bilgileri kullanıcıya yerel olarak göstermektir. Bu nedenle OLED ekranın bağlantıları, proje kutusunun ön yüzüne yerleştirilebilecek şekilde planlanmıştır. Benzer şekilde IR alıcı modülü de uzaktan kumandadan gelen sinyalleri algılayabilecek uygun bir konuma yerleştirilmiştir. Buton ise kullanıcının sisteme doğrudan fiziksel müdahale edebilmesi için erişilebilir bir noktada konumlandırılmıştır.

Pertinaks üzerindeki kablo düzeninde güç, toprak ve sinyal hatlarının karışmamasına dikkat edilmiştir. Devrede tüm modüllerin ortak GND hattına bağlanması sağlanmış, böylece ESP32, röle modülleri, sensörler ve ekran arasında ortak referans noktası oluşturulmuştur. Güç bağlantılarında ise kullanılan modüllerin çalışma gerilimleri dikkate alınmış ve bağlantılar buna göre düzenlenmiştir. Özellikle röle modülleri ve fan gibi çıkış birimlerinin besleme hatları, sensör ve haberleşme hatlarından ayrı düşünülerek daha güvenli bir bağlantı yapısı elde edilmiştir.

Pertinaks üzerindeki donanım yerleşimi yapılırken sistemin test edilebilir ve bakım yapılabilir olmasına da dikkat edilmiştir. ESP32'nin USB bağlantı noktası erişilebilir olacak şekilde konumlandırılmış, böylece yazılım güncelleme ve seri port üzerinden hata takibi işlemlerinin kolay yapılabilmesi sağlanmıştır. Sensör, röle ve ekran bağlantıları gerektiğinde kontrol edilebilecek şekilde düzenlenmiş, kabloların birbirine temas ederek kısa devre oluşturmasını engellemek amacıyla bağlantı noktalarının sabit ve düzenli olmasına özen gösterilmiştir.

Sonuç olarak pertinaks üzerindeki donanım yerleşimi, sistemin breadboard aşamasından daha kararlı ve düzenli bir yapıya geçmesini sağlamıştır. ESP32 merkezli kontrol yapısı, sensör girişleri, röle çıkışları, OLED ekran, IR alıcı ve buton bağlantıları tek bir fiziksel düzen içerisinde birleştirilmiştir. Bu yerleşim sayesinde sistem, proje kutusuna monte edilebilecek kompakt ve bütünleşik bir donanım yapısına dönüştürülmüştür.



Şekil 5.2.3 Pertinaks Görünümü

5.2.5. Güç Beslemesi ve Kablo Düzeni

Projede kullanılan donanım bileşenlerinin kararlı şekilde çalışabilmesi için güç beslemesi ve kablo düzeni önemli bir tasarım aşaması olarak ele alınmıştır. Sistemde ESP32 mikrodenetleyicisi, sensörler, OLED ekran, röle modülleri ve fan gibi farklı çalışma özelliklerine sahip bileşenler bulunduğundan, her bir elemanın uygun gerilim seviyesinde beslenmesine dikkat edilmiştir. ESP32 mikrodenetleyicisi sistemin ana kontrol birimi olarak görev yaptığı için diğer düşük gerilimli modüller ile ortak referans hattı üzerinden çalışacak şekilde bağlantılandırılmıştır.

DHT11 sıcaklık ve nem sensörü, LDR sensör modülü, IR alıcı modülü, buton ve OLED ekran gibi düşük güçlü bileşenler ESP32 ile uyumlu olacak şekilde beslenmiştir. OLED ekran, ESP32 ile I2C haberleşme hattı üzerinden bağlantı kurarken; DHT11, LDR, IR alıcı ve buton ise ilgili giriş pinlerine bağlanmıştır. Bu bileşenlerin sağlıklı veri iletebilmesi için besleme ve toprak hatlarının düzenli şekilde oluşturulmasına dikkat edilmiştir.

Sistemde lamba ve fan gibi çıkış birimlerinin kontrolü röle modülleri aracılığıyla gerçekleştirilmiştir. Röleler, ESP32'den gelen düşük gerilimli kontrol sinyali ile tetiklenmekte ve buna bağlı olarak yüklerin açma-kapama işlemlerini gerçekleştirmektedir. Fan, düşük gerilimli bir DC yük olarak sisteme dâhil edilmiş ve röle üzerinden kontrol edilecek şekilde yapılandırılmıştır. Lamba bağlantısında ise yük hattı röle kontakları üzerinden anahtarlanmıştır. Böylece ESP32 doğrudan yüksek akım veya yüksek gerilim taşıyan yük hattına bağlanmadan, röle aracılığıyla güvenli bir kontrol yapısı oluşturulmuştur.

Kablo düzeninde güç, toprak ve sinyal hatlarının birbirinden ayırt edilebilir şekilde düzenlenmesine özen gösterilmiştir. Tüm düşük gerilimli devre elemanlarının ortak GND hattına bağlanması sağlanmış, böylece ESP32, sensörler, OLED ekran ve röle modülleri arasında ortak referans noktası oluşturulmuştur. Ortak GND hattı, özellikle sensör verilerinin doğru okunması ve röle kontrol sinyallerinin kararlı çalışması açısından önemlidir.

Sinyal kabloları ile güç kablolarının karışmaması için bağlantılar mümkün olduğunca düzenli bir şekilde yerleştirilmiştir. Röle modüllerine giden kontrol sinyalleri, sensör bağlantıları ve OLED ekranın haberleşme hatları takip edilebilir olacak şekilde konumlandırılmıştır. Bu düzen, hem devrenin test edilmesini kolaylaştırmış hem de olası bağlantı hatalarının daha hızlı tespit edilmesini sağlamıştır.

Bağlantı güvenliği açısından kabloların gevşek kalmamasına, kısa devre oluşturabilecek açık uçların izole edilmesine ve yük bağlantılarının sağlam şekilde yapılmasına dikkat edilmiştir. Özellikle röle üzerinden kontrol edilen lamba bağlantısında güvenlik ön planda tutulmuş, yüksek gerilim hattı ile düşük gerilimli kontrol devresinin birbirinden ayrılmasına özen gösterilmiştir. Bu sayede sistemin hem güvenli hem de kararlı şekilde çalışması hedeflenmiştir.

Sonuç olarak güç beslemesi ve kablo düzeni, sistemin güvenilir çalışması için donanım tasarımının önemli bir parçası olmuştur. Ortak GND yapısı, düzenli kablo bağlantıları, röleler üzerinden yük kontrolü ve düşük gerilimli kontrol devresinin ayrı tutulması sayesinde sistem daha güvenli, düzenli ve test edilebilir bir yapıya kavuşturulmuştur.

5.2.7. Kutulama Tasarımı ve Donanım Yerleşiminin Değerlendirilmesi

Projenin donanım kurulumunun tamamlanmasının ardından sistemin daha düzenli, taşınabilir, güvenli ve kullanıcı tarafından kolay erişilebilir bir yapıya sahip olması amacıyla kutulama tasarımı gerçekleştirilmiştir. Kutulama tasarımında temel amaç, devre elemanlarının proje kutusu içerisine uygun şekilde yerleştirilmesi, kullanıcı ile etkileşimde bulunacak bileşenlerin dış yüzeye alınması ve bağlantı hatlarının güvenli bir düzende tutulmasıdır. Bu kapsamda ESP32 mikrodenetleyicisi, röle modülleri, sensör bağlantıları, güç hatları ve haberleşme bağlantıları kutu içerisinde konumlandırılmıştır. Böylece sistemin açık devre yapısından çıkarılarak daha bütünleşik ve korunabilir bir forma dönüştürülmesi sağlanmıştır.

Kutu içerisinde yer alan devre elemanları, sistemin çalışma mantığına uygun şekilde yerleştirilmiştir. ESP32 mikrodenetleyicisi, sistemin ana kontrol birimi olduğu için sensörler, röle modülleri, OLED ekran, IR alıcı ve buton bağlantılarına erişebilecek merkezi bir konumda tutulmuştur. Röle modülleri ise lamba ve fan gibi çıkış birimlerini kontrol ettiği için yük bağlantılarına yakın olacak şekilde konumlandırılmıştır. Bu yerleşim sayesinde hem kablo uzunlukları azaltılmış hem de bağlantıların daha düzenli takip edilebilmesi sağlanmıştır.

OLED ekran, sistem durumlarının kullanıcı tarafından kolayca görülebilmesi amacıyla kutunun ön yüzeyine yerleştirilmiştir. Ekran üzerinden sıcaklık, nem, ışık seviyesi, lamba durumu ve fan durumu gibi bilgiler anlık olarak takip edilebilmektedir. Bu sayede kullanıcı, Node-RED Dashboard veya mobil uygulamaya ihtiyaç duymadan sistemin temel çalışma durumunu doğrudan cihaz üzerinden gözlemleyebilmektedir. OLED ekranın dış yüzeye yerleştirilmesi, sistemin kullanıcı arayüzü açısından daha işlevsel bir yapıya sahip olmasını sağlamıştır.

Buton, kullanıcının sisteme fiziksel olarak müdahale edebilmesi amacıyla kolay erişilebilir bir noktaya konumlandırılmıştır. Bu sayede internet bağlantısı, mobil uygulama veya web arayüzü kullanılmadan da yerel kontrol işlemi gerçekleştirilebilmektedir. Butonun ön yüzeyde yer alması, sistemin yalnızca uzaktan kontrol edilen bir yapı olmasını engellemiş ve kullanıcıya doğrudan manuel kontrol imkânı sağlamıştır.

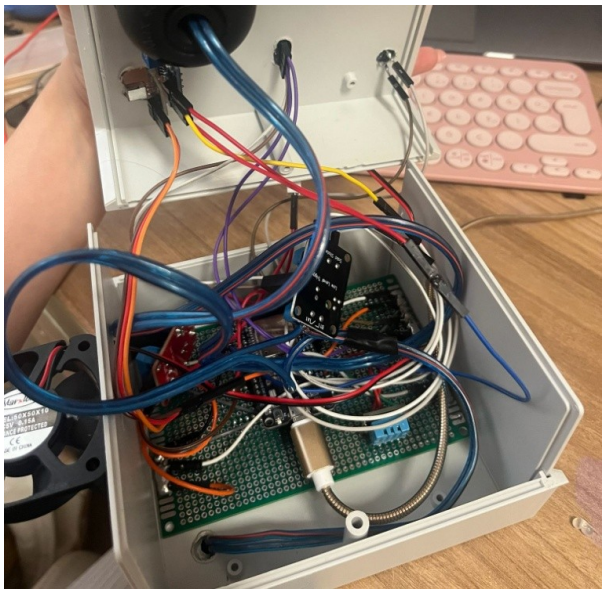
IR alıcı modülü, uzaktan kumandadan gelen kızılötesi sinyalleri algılayabilecek şekilde kutunun dış yüzeye yakın bir bölümüne yerleştirilmiştir. IR sinyallerinin sağlıklı şekilde algılanabilmesi için alıcı modülün önünün kapatılmamasına ve kumanda ile görüş hattının mümkün olduğunca korunmasına dikkat edilmiştir. Bu yerleşim sayesinde kullanıcı, IR uzaktan kumanda aracılığıyla lamba ve fan kontrolünü kolaylıkla gerçekleştirebilmektedir.

LDR sensör modülü, ortam ışık seviyesini doğru algılayabilmesi için dış ortam ışığını görebilecek uygun bir konuma yerleştirilmiştir. Sensörün tamamen kutu içerisinde kalması durumunda ortam aydınlık seviyesi doğru ölçülemeyeceğinden, LDR algılama yüzeyi dış ortama açık olacak şekilde konumlandırılmıştır. Böylece ortamın aydınlık veya karanlık durumunun daha doğru şekilde algılanması amaçlanmıştır.

Fan, hava akışını sağlayabilecek şekilde kutu üzerinde uygun bir açıklığa yerleştirilmiştir. Fanın konumu belirlenirken hava çıkışının engellenmemesine, fan kanatlarının kablo bağlantılarına temas etmemesine ve kutu içerisinde oluşabilecek ısı birikiminin azaltılmasına dikkat edilmiştir. Fanın dış yüzeye yakın yerleştirilmesi, hem hava sirkülasyonunu artırmış hem de sistemin havalandırma görevini daha verimli şekilde yerine getirmesini sağlamıştır.

Güç girişi, sistemin besleme bağlantısının güvenli ve düzenli yapılabilmesi amacıyla kutunun uygun bir yan veya arka bölümüne konumlandırılmıştır. Bu yerleşim sayesinde dışarıdan gelen güç kablosunun kutu içerisinde gereksiz kablo karmaşası oluşturmasının önüne geçilmiştir. Ayrıca güç hattı ile sinyal hatlarının mümkün olduğunca ayrı tutulmasına dikkat edilerek bağlantı güvenliği artırılmıştır. Özellikle röle modülleri üzerinden kontrol edilen yük bağlantılarında, düşük gerilimli kontrol devresi ile yük hattının birbirinden ayrılması tasarımda önemli bir güvenlik unsuru olarak ele alınmıştır.

Kutu içerisindeki kablo düzeninde güç, toprak ve sinyal hatlarının takip edilebilir olmasına özen gösterilmiştir. Tüm düşük gerilimli devre elemanlarının ortak GND hattı üzerinden çalışması sağlanmış, böylece ESP32, sensörler, OLED ekran ve röle modülleri arasında ortak referans noktası oluşturulmuştur. Ortak GND yapısı, sensör verilerinin doğru okunması ve kontrol sinyallerinin kararlı çalışması açısından önemlidir. Ayrıca bağlantıların daha güvenli hâle getirilmesi için kabloların gevşek kalmamasına, fan kanatlarına temas etmemesine ve açıkta kalan iletken uçların izole edilmesine dikkat edilmiştir.Bağlantı noktalarında oluşabilecek temas, gevşeme ve kısa devre risklerini azaltmak amacıyla bazı kablo uçları ve hassas bağlantı bölgeleri silikon ile sabitlenmiştir. Bu işlem sayesinde kabloların kutu içerisinde hareket etmesi engellenmiş, bağlantıların mekanik dayanımı artırılmış ve sistemin daha güvenli şekilde çalışması hedeflenmiştir. Özellikle sensör bağlantıları, güç hattı geçişleri ve kutu yüzeyine yakın kablo bölgelerinde yapılan sabitleme işlemleri, sistemin taşınması veya çalışması sırasında oluşabilecek bağlantı problemlerini azaltmıştır.Genel olarak kutulama tasarımı, sistemin yalnızca çalışır durumda olmasını değil, aynı zamanda düzenli, taşınabilir, güvenli ve test edilebilir bir yapıya sahip olmasını sağlamıştır. Pertinaks üzerine kurulan devre, proje kutusu içerisine yerleştirilerek breadboard aşamasındaki geçici bağlantı yapısından daha kararlı bir düzene geçirilmiştir. Kullanıcı tarafından erişilmesi gereken OLED ekran, buton, IR alıcı, LDR sensör ve fan gibi bileşenler dış yüzeye uygun şekilde konumlandırılmış; kontrol ve haberleşme elemanları ise kutu içerisinde korunacak şekilde yerleştirilmiştir. Bu tasarım sayesinde proje, hem akademik sunum hem de uygulamalı kullanım açısından daha bütünleşik bir donanım yapısına dönüştürülmüştür.



Şekil 5.2.7.1 Projenin İç Görüntüsü

Şekil 5.2.7.1’de proje kutusunun içten görünümü verilmiştir. Görselde sistemde kullanılan pertinaks kartın kutu içerisine yerleştirildiği, ESP32 mikrodenetleyicisi, sensör bağlantıları, röle bağlantıları ve güç hatlarının aynı yapı üzerinde toplandığı görülmektedir. Kutu içerisinde bağlantı kabloları, sensörlerin dış ortamla temas edebileceği açıklıklara yönlendirilmiş; fan, buton, OLED ekran ve güç girişi gibi dışarıdan erişilmesi gereken bileşenler kutu yüzeylerine uygun şekilde konumlandırılmıştır. Bu yerleşim sayesinde sistemin taşınabilir, test edilebilir ve bütünleşik bir yapıya sahip olması amaçlanmıştır.



Şekil 5.2.7.2 Projenin Dış Görüntüsü

Şekil 5.2.7.2’de projenin dış görünümü verilmiştir. Görselde IoT tabanlı akıllı priz ve ortam izleme sisteminin proje kutusu içerisine yerleştirilmiş son hâli görülmektedir. Donanım elemanlarının kutu içerisine alınmasıyla birlikte sistem, breadboard veya açık bağlantı düzeninden çıkarılarak daha düzenli, taşınabilir ve kullanıcıya uygun bir yapıya dönüştürülmüştür. Kutulama tasarımında, kullanıcı tarafından doğrudan erişilmesi veya gözlemlenmesi gereken bileşenlerin dış yüzeyde konumlandırılmasına dikkat edilmiştir. Bu kapsamda lamba, OLED ekran, IR alıcı bölümü, LDR sensör ve buton kutunun ön yüzeyine yerleştirilmiştir.

OLED ekran, sistemin anlık çalışma durumlarının kullanıcı tarafından kolayca takip edilebilmesi amacıyla kutunun ön yüzeyinde konumlandırılmıştır. Bu ekran üzerinden sıcaklık, nem, ışık seviyesi, lamba durumu ve fan durumu gibi bilgiler kullanıcıya görsel olarak sunulabilmektedir. Böylece kullanıcı, yalnızca Node-RED Dashboard veya mobil uygulama üzerinden değil, doğrudan cihaz üzerindeki ekran aracılığıyla da sistemin mevcut durumunu izleyebilmektedir. Buton ise yerel kontrol işlemlerinin fiziksel olarak gerçekleştirilebilmesi için erişilebilir bir noktaya yerleştirilmiştir. Bu sayede internet bağlantısı veya uzaktan kontrol arayüzü kullanılmadan da sistem üzerinde manuel kontrol yapılabilmektedir.

IR alıcı bölümü, uzaktan kumandadan gelen kızılötesi sinyalleri algılayabilecek şekilde kutunun ön yüzeyinde uygun bir açıklığa yerleştirilmiştir. Bu yerleşim sayesinde kumanda ile gönderilen kontrol sinyallerinin alıcı modüle daha sağlıklı ulaşması amaçlanmıştır. LDR sensör ise ortam ışık seviyesini doğru algılayabilmesi için dış ortam ışığını görebilecek bir

noktada konumlandırılmıştır. Sensörün kutu içerisine tamamen kapatılmaması, ortamın aydınlık veya karanlık durumunun daha doğru şekilde ölçülmesine katkı sağlamaktadır. Fan ise hava akışını sağlayacak şekilde kutunun üst kısmına yerleştirilmiştir. Fanın bu bölgede konumlandırılması, hem hava çıkışının engellenmemesini hem de sistemin havalandırma işlevini daha verimli şekilde yerine getirmesini sağlamaktadır.

Kutulama sırasında yalnızca bileşenlerin yerleşimine değil, bağlantı güvenliğine de dikkat edilmiştir. Kablo bağlantılarının fan kanatlarına temas etmemesi, güç ve sinyal hatlarının mümkün olduğunca düzenli tutulması ve açıkta kalan bağlantı noktalarının risk oluşturmayacak şekilde konumlandırılması hedeflenmiştir. Bağlantı noktalarında oluşabilecek temas, gevşeme ve kısa devre ihtimalini azaltmak amacıyla bazı kablo uçları ve hassas bağlantı bölgeleri silikon ile sabitlenmiştir. Bu işlem sayesinde kabloların kutu içerisinde hareket etmesi sınırlandırılmış, bağlantıların mekanik dayanımı artırılmış ve sistemin daha güvenli çalışması amaçlanmıştır.

Genel olarak projenin dış görünümünde, kullanıcı arayüzü elemanlarının erişilebilir, sensörlerin ölçüm yapmaya uygun ve çıkış birimlerinin işlevsel olacak şekilde yerleştirildiği görülmektedir. Bu kutulama tasarımı sayesinde sistem, yalnızca çalışır bir devre olmaktan çıkarılarak sunuma ve uygulamaya uygun, taşınabilir ve bütünleşik bir IoT cihazı hâline getirilmiştir.

5.3 Yazılım Tasarımı

Bu bölümde, IoT tabanlı akıllı priz ve ortam izleme sisteminin yazılım tasarım süreci açıklanmaktadır. Projenin yazılım yapısı, ESP32 mikrodenetleyicisi üzerinde çalışan kontrol yazılımı ile Raspberry Pi üzerinde çalışan Mosquitto MQTT Broker ve Node-RED akışlarından oluşmaktadır. ESP32 tarafında sensör verilerinin okunması, röle modüllerinin kontrol edilmesi, OLED ekrana sistem bilgilerinin yazdırılması, IR kumanda ve fiziksel buton girişlerinin algılanması gibi işlemler gerçekleştirilmiştir. Raspberry Pi tarafında ise MQTT haberleşmesi, Node-RED akış yönetimi, Dashboard arayüzü, ThingSpeak veri aktarımı ve Telegram bildirim yapısı oluşturulmuştur.

Yazılım tasarımında temel amaç, sistemde kullanılan tüm donanım bileşenlerinin birbiriyle uyumlu, kararlı ve senkronize şekilde çalışmasını sağlamaktır. Bu kapsamda ESP32, sistemin saha tarafındaki ana kontrol birimi olarak görev yapmaktadır. DHT11 sıcaklık ve nem sensöründen alınan veriler, LDR sensör modülünden elde edilen ortam ışık bilgisi, IR kumandadan gelen kontrol sinyalleri ve fiziksel buton durumu ESP32 tarafından işlenmektedir. Elde edilen verilere ve kullanıcıdan gelen komutlara göre lamba ve fan röleleri kontrol edilmekte, sistemin anlık çalışma durumu OLED ekran üzerinde kullanıcıya gösterilmektedir. Böylece yazılım, hem veri toplama hem de cihaz kontrolü görevlerini aynı yapı içerisinde gerçekleştirmektedir.

Sistemde haberleşme altyapısı MQTT protokolü üzerine kurulmuştur. ESP32, Wi-Fi bağlantısı aracılığıyla Raspberry Pi üzerinde çalışan Mosquitto MQTT Broker'a bağlanmaktadır. Node-RED Dashboard veya mobil MQTT uygulaması üzerinden gönderilen lamba ve fan kontrol komutları, belirlenen MQTT topic yapıları üzerinden ESP32'ye iletilmektedir. ESP32, ilgili topic'lere abone olarak gelen komutları almakta ve mesaj içeriğine göre lamba veya fan rölesini açıp kapatmaktadır. Aynı zamanda ESP32 tarafından okunan sıcaklık, nem, ışık seviyesi, lamba durumu ve fan durumu gibi bilgiler MQTT üzerinden yayınlanmaktadır. Yayınlanan bu veriler Node-RED tarafından alınarak kullanıcı arayüzünde gösterilmekte ve gerekli durumlarda ThingSpeak platformuna aktarılmaktadır.

Node-RED, yazılım tasarımında merkezi kontrol ve veri yönlendirme platformu olarak kullanılmıştır. Node-RED üzerinde oluşturulan akışlar sayesinde MQTT broker üzerinden gelen sensör verileri işlenmiş, Dashboard arayüzüne aktarılmış ve ThingSpeak platformuna gönderilmiştir. Ayrıca lamba ve fan kontrolü için Dashboard üzerinde oluşturulan anahtar bileşenleri, MQTT topic yapılarıyla ilişkilendirilmiştir. Kullanıcı Dashboard üzerinden bir kontrol işlemi gerçekleştirdiğinde, Node-RED bu komutu MQTT broker'a iletmekte ve ESP32 ilgili komutu alarak çıkış birimini kontrol etmektedir. Bu yapı sayesinde sistem, web tabanlı bir arayüz üzerinden izlenebilir ve kontrol edilebilir hâle getirilmiştir.

Yazılım tasarımında yerel kontrol yöntemleri de dikkate alınmıştır. Sistemde kullanılan fiziksel buton, kullanıcının doğrudan cihaz üzerinden lamba kontrolü yapabilmesini sağlamaktadır. IR uzaktan kumanda ise kullanıcıya kablosuz yerel kontrol imkânı sunmaktadır. ESP32 yazılımında IR kumandadan gelen tuş kodları tanımlanmış ve bu kodlara göre lamba ile fanın açma-kapama işlemleri gerçekleştirilmiştir. Bu sayede sistem yalnızca internet veya mobil uygulama üzerinden kontrol edilen bir yapı olmaktan çıkarılmış, aynı zamanda yerel ve kablosuz kontrol seçenekleriyle desteklenmiştir.

OLED ekran, yazılım tasarımında kullanıcı bilgilendirme arayüzü olarak değerlendirilmiştir. ESP32 tarafından işlenen MQTT bağlantı durumu, lamba ve fanın çalışma durumu, sıcaklık,

nem ve ortam ışık bilgisi OLED ekran üzerinde gösterilmektedir. Bu yapı sayesinde kullanıcı, Node-RED Dashboard veya mobil uygulamaya erişmeden de sistemin anlık durumunu cihaz üzerinden takip edebilmektedir. OLED ekranın her durum değişikliğinde güncellenmesi, sistemin kullanıcıya daha anlaşılır ve takip edilebilir bir çalışma yapısı sunmasını sağlamıştır.

Genel olarak yazılım tasarımı; sensör okuma, cihaz kontrolü, veri yayınlama, kullanıcı arayüzü oluşturma, bulut tabanlı veri izleme ve bildirim işlemlerinin bir arada çalışabileceği modüler bir yapıda oluşturulmuştur. ESP32 yazılımında temel işlemler ayrı fonksiyonlar hâlinde ele alınmış, böylece kodun okunabilirliği ve geliştirilebilirliği artırılmıştır. Node-RED tarafında ise görsel akış yapısı kullanılarak verilerin hangi kaynaklardan alındığı, hangi işlemlerden geçtiği ve hangi platformlara gönderildiği açık şekilde düzenlenmiştir. Bu sayede proje, hem yerel hem de uzaktan kontrol edilebilen, sensör verilerini izleyebilen ve farklı yazılım platformlarıyla entegre çalışan bütünleşik bir IoT yazılım mimarisine sahip olmuştur.

5.3.1. MQTT Mesaj İşleme Yapısı

ESP32 yazılımında MQTT üzerinden gelen komutların işlenmesi için callback fonksiyonu kullanılmıştır. Node-RED Dashboard veya mobil MQTT uygulaması tarafından gönderilen komutlar, belirlenen topic yapıları üzerinden ESP32'ye ulaşmaktadır. ESP32, gelen mesajın hangi topic üzerinden geldiğini kontrol etmekte ve mesaj içeriğine göre lamba veya fan durumunu değiştirmektedir. Bu yapıda "1" mesajı ilgili çıkış birimini açmak, "0" mesajı ise kapatmak için kullanılmıştır.

```
void callback(char* topic, byte* payload, unsigned int length) {
  String mesaj = "";

  for (unsigned int i = 0; i < length; i++) {
    mesaj += (char)payload[i];
  }

  mesaj.trim();

  Serial.print("MQTT mesaj geldi [");
  Serial.print(topic);
  Serial.print("] : ");
  Serial.println(mesaj);

  if (String(topic) == topic_lamba_cmd) {
    if (mesaj == "1") {
      setLamp(true);
    } else if (mesaj == "0") {
      setLamp(false);
    }
  }

  if (String(topic) == topic_fan_cmd) {
    if (mesaj == "1") {
      setFan(true);
    } else if (mesaj == "0") {
      setFan(false);
    }
```

Şekil 5.3.1 MQTT Mesaj İşleme

5.3.2. Wi-Fi ve MQTT Bağlantı Yapısı

ESP32'nin Raspberry Pi üzerinde çalışan MQTT broker ile haberleşebilmesi için öncelikle Wi-Fi ağına bağlanması gerekmektedir. Yazılımda Wi-Fi bağlantısı kurulduktan sonra MQTT sunucu adresi ve port bilgisi kullanılarak Mosquitto MQTT Broker'a bağlantı sağlanmaktadır. MQTT bağlantısı koptuğunda sistem yeniden bağlanmayı denemekte ve bağlantı durumu OLED ekran üzerinde kullanıcıya gösterilmektedir. Bu yapı, sistemin haberleşme sürekliliğini koruması açısından önemlidir. Ayrıca bağlantı durumunun hem Serial Monitor üzerinden hem de OLED ekran aracılığıyla takip edilebilmesi, hata tespiti ve sistem kontrolünü kolaylaştırmaktadır. Böylece Wi-Fi veya MQTT bağlantısında oluşabilecek kesintiler

kullanıcı tarafından hızlı şekilde fark edilebilmekte ve sistemin çalışma durumu izlenebilmektedir.

```
void connectWiFi() {
  Serial.print("WiFi baglaniyor: ");
  Serial.println(ssid);

  mqttDurum = "WIFI YOK";
  oledYazdir();

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi baglandi");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());

  mqttDurum = "BAGLANMIYOR";
  oledYazdir();
}
```

5.3.2. Wi-Fi ve MQTT Bağlantı Yapısı

5.3.3. Ana Döngü, Buton ve IR Kumanda Kontrolü

ESP32 yazılımının loop bölümünde sistemin sürekli çalışması sağlanmaktadır. Bu bölümde Wi-Fi ve MQTT bağlantıları kontrol edilmekte, bağlantı kopması durumunda yeniden bağlantı kurulmaktadır. MQTT mesajlarının işlenmesi için client.loop fonksiyonu çalıştırılmakta, sensör verileri düzenli olarak okunarak yayınlanmaktadır. Ayrıca fiziksel buton ile lamba kontrolü ve IR uzaktan kumanda ile lamba ve fan kontrolü bu bölümde gerçekleştirilmiştir. Buton kontrolünde istenmeyen tetiklemeleri azaltmak amacıyla kısa bir debounce gecikmesi kullanılmıştır.

```
void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
  }

  if (!client.connected()) {
    reconnectMQTT();
  } else {
    mqttDurum = "BAGLANDI";
  }

  client.loop();

  publishSensorData();

  static int lastButtonState = HIGH;
  int buttonState = digitalRead(BUTTON_PIN);

  if (lastButtonState == HIGH && buttonState == LOW) {
    delay(30);
    if (digitalRead(BUTTON_PIN) == LOW) {
      toggleLamp();
      while (digitalRead(BUTTON_PIN) == LOW) {
      }
      delay(30);
    }
  }
```

5.3.3. Ana Döngü, Buton ve IR Kumanda Kontrolü

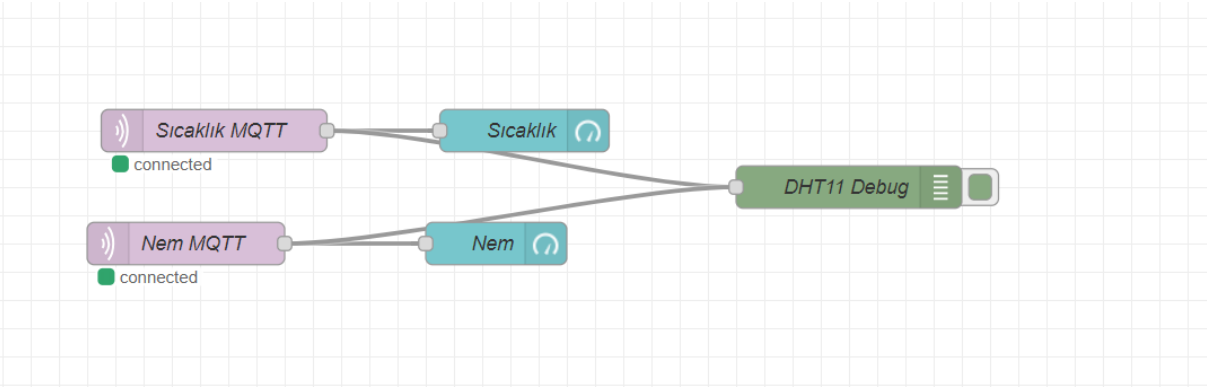
5.3.4. ESP32 Yazılımının Genel Değerlendirmesi

ESP32 yazılımı, sistemdeki donanım bileşenlerini merkezi olarak yönetecek şekilde tasarlanmıştır. Yazılım içerisinde sensör okuma, röle kontrolü, OLED ekran güncelleme, MQTT haberleşmesi, IR kumanda ve buton kontrolü ayrı fonksiyonlar hâlinde ele alınmıştır. Bu yapı, yazılımın daha anlaşılır, düzenli ve geliştirilebilir olmasını sağlamıştır. ESP32, yerel kontrol elemanlarından gelen komutları işleyebilmekte, MQTT üzerinden uzaktan kontrol komutlarını alabilmekte ve sensör verilerini düzenli olarak yayınlayabilmektedir. Böylece sistem hem yerel hem de uzaktan kontrol edilebilen bütünleşik bir IoT yazılım mimarisine sahip olmuştur.

5.3.5. Node-RED Akış Tasarımı

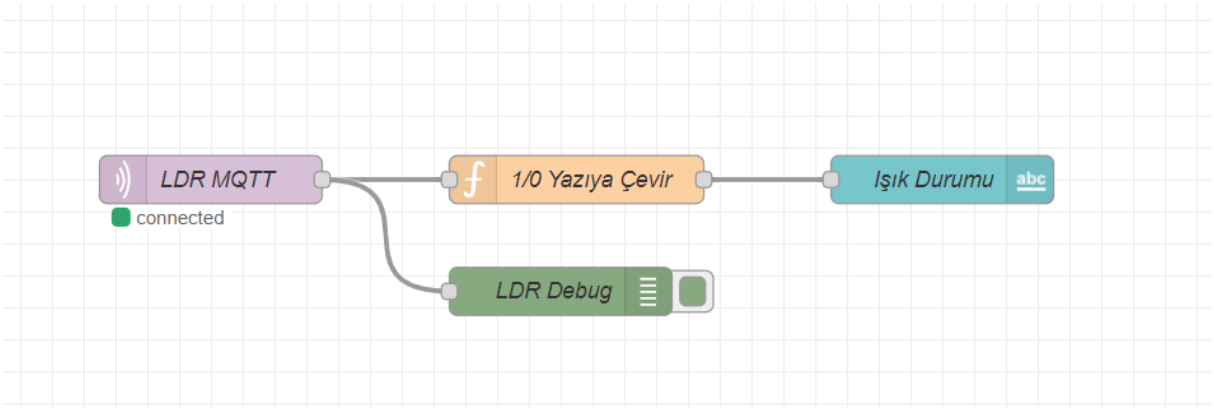
Projede Node-RED, ESP32 ile kullanıcı arayüzleri, ThingSpeak platformu ve Telegram bildirimi arasında veri akışını yönetmek amacıyla kullanılmıştır. Raspberry Pi üzerinde çalışan Node-RED, Mosquitto MQTT Broker üzerinden ESP32'den gelen verileri almakta ve kullanıcı tarafından oluşturulan kontrol komutlarını yine MQTT protokolü aracılığıyla ESP32'ye göndermektedir. Bu yapı sayesinde sistemin kontrol, izleme, veri kaydetme ve bildirim işlemleri merkezi bir yazılım akışı üzerinden gerçekleştirilmiştir.

Node-RED akış tasarımında sıcaklık, nem, ışık durumu, lamba durumu ve fan durumu gibi veriler MQTT giriş düğümleri ile alınmıştır. Alınan veriler gerekli durumlarda function düğümleri ile işlenmiş, Dashboard bileşenlerine aktarılmış, ThingSpeak platformuna gönderilmiş veya Telegram mesajı oluşturmak için kullanılmıştır. Ayrıca lamba ve fan kontrolü için Dashboard ve inject düğümleri üzerinden MQTT komutları oluşturulmuş, bu komutlar ESP32'ye gönderilerek rölelerin kontrol edilmesi sağlanmıştır.



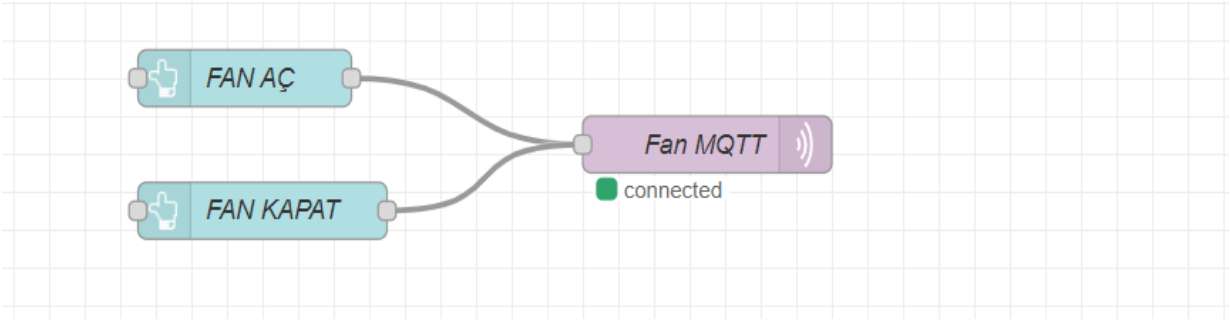
Şekil 5.3.5.1 DHT11 sıcaklık ve nem verisi NODE-RED akışı

Şekil 5.3.5.1'te DHT11 sıcaklık ve nem sensöründen gelen verilerin Node-RED üzerinde işlenmesine ait akış verilmiştir. Bu akışta "Sıcaklık MQTT" ve "Nem MQTT" düğümleri, ESP32 tarafından MQTT üzerinden yayınlanan sıcaklık ve nem topic'lerini dinlemektedir. Alınan veriler Dashboard üzerinde oluşturulan sıcaklık ve nem göstergelerine aktarılmıştır. Ayrıca verilerin doğru şekilde alınıp alınmadığını kontrol etmek amacıyla debug düğümü kullanılmıştır. Bu yapı sayesinde ortam sıcaklığı ve nem değerleri web tabanlı arayüz üzerinden anlık olarak izlenebilmiştir.



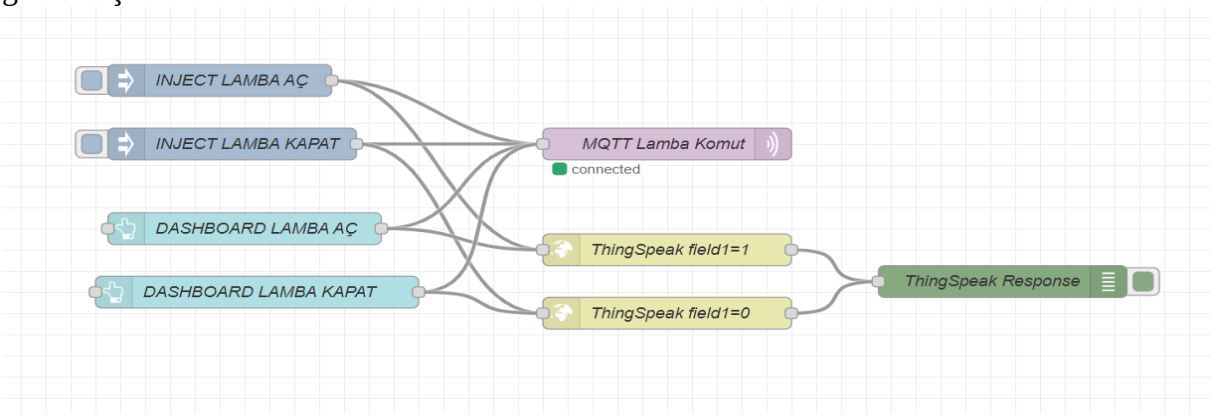
Şekil 5.3.5.2 LDR ışık durumu Node-RED akışı

Şekil 5.3.5.2'te LDR sensöründen gelen ışık durumu verisinin Node-RED üzerindeki akışı gösterilmiştir. LDR sensöründen alınan veri ESP32 tarafından MQTT üzerinden Node-RED'e gönderilmektedir. Node-RED üzerinde yer alan "LDR MQTT" düğümü bu veriyi almakta, ardından function düğümü aracılığıyla 0 ve 1 şeklindeki sayısal değerler kullanıcı için daha anlaşılır olan metinsel ifadelere dönüştürülmektedir. Bu dönüşüm sonucunda ortam durumu "Aydınlık" veya "Karanlık" olarak Dashboard üzerinde gösterilmektedir. Debug düğümü ise akışın test edilmesi ve gelen verinin kontrol edilmesi amacıyla kullanılmıştır.



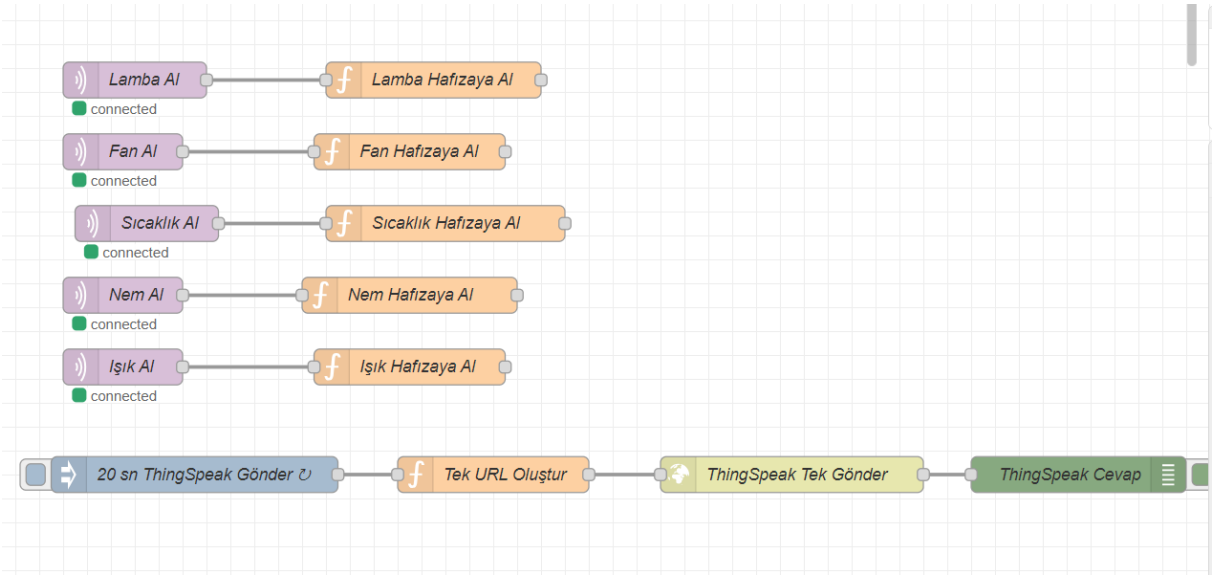
Şekil 5.3.5.3 Fan kontrol Node-RED akışı

Şekil 5.3.5.3'te fan kontrolü için oluşturulan Node-RED akışı verilmiştir. Bu akışta Dashboard üzerinden oluşturulan "FAN AÇ" ve "FAN KAPAT" kontrol düğümleri, kullanıcı tarafından verilen fan kontrol komutlarını üretmektedir. Oluşturulan komutlar "Fan MQTT" çıkış düğümüne iletilmekte ve ilgili MQTT topic'i üzerinden ESP32'ye gönderilmektedir. ESP32 gelen komutu işleyerek fan rölesini aktif veya pasif duruma getirmektedir. Böylece fan, Node-RED Dashboard üzerinden uzaktan kontrol edilebilen bir çıkış birimi hâline getirilmiştir.



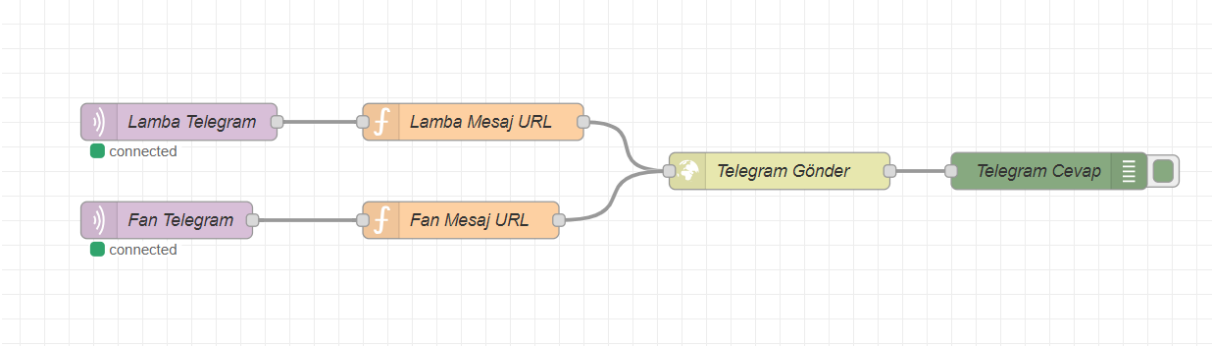
Şekil 5.3.5.4 Lamba kontrol NODE-RED akışı

Şekil 5.3.5.4'te lamba kontrolü ve lamba durumunun ThingSpeak platformuna aktarılması için oluşturulan Node-RED akışı gösterilmiştir. Akışta "INJECT LAMBA AÇ" ve "INJECT LAMBA KAPAT" düğümleri test amacıyla kullanılmıştır. Bunun yanında Dashboard üzerinden oluşturulan "DASHBOARD LAMBA AÇ" ve "DASHBOARD LAMBA KAPAT" düğümleri, kullanıcının web arayüzü üzerinden lambayı kontrol etmesini sağlamaktadır. Bu komutlar "MQTT Lamba Komut" düğümü aracılığıyla ESP32'ye gönderilmekte ve ESP32 lamba rölesini açma veya kapama işlemini gerçekleştirmektedir. Aynı akış içerisinde lamba durumunun ThingSpeak'e gönderilmesi için field değerleri oluşturulmuş, böylece lambanın açık veya kapalı durumu bulut platformunda izlenebilir hâle getirilmiştir.



Şekil 5.3.5.5 ThingSpeak veri gönderme Node-RED akışı

Şekil 5.3.5.5'te sistem verilerinin ThingSpeak platformuna gönderilmesine ait Node-RED akışı verilmiştir. Bu akışta lamba, fan, sıcaklık, nem ve ışık bilgileri MQTT giriş düğümleri ile alınmaktadır. Alınan değerler function düğümleri aracılığıyla hafızaya kaydedilmekte ve ThingSpeak'e gönderilecek veri yapısı için hazır hâle getirilmektedir. "20 sn ThingSpeak Gönder" düğümü belirli zaman aralıklarında akışı tetiklemekte, "Tek URL Oluştur" düğümü ise ThingSpeak API yapısına uygun URL oluşturmaktadır. Oluşturulan URL, HTTP request düğümü ile ThingSpeak platformuna gönderilmekte ve "ThingSpeak Cevap" debug düğümü ile gönderim sonucu kontrol edilmektedir. Bu yapı sayesinde sistemden alınan veriler düzenli aralıklarla ThingSpeak üzerinde grafiksel olarak takip edilebilmiştir.



Şekil 5.3.5.6 Telegram bildirim Node-RED akışı

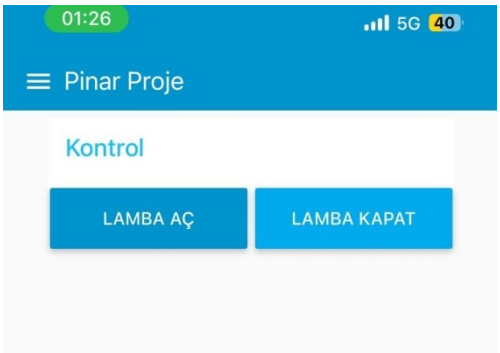
Şekil 5.3.5.6'te lamba ve fan durumlarına bağlı Telegram bildirim akışı gösterilmiştir. Node-RED üzerinde "Lamba Telegram" ve "Fan Telegram" MQTT giriş düğümleri, sistemden gelen lamba ve fan durum bilgilerini almaktadır. Bu bilgiler function düğümleri aracılığıyla Telegram mesaj formatına dönüştürülmektedir. Daha sonra "Telegram Gönder" HTTP request düğümü kullanılarak Telegram bot API'si üzerinden kullanıcıya mesaj gönderilmektedir. "Telegram Cevap" debug düğümü ise mesaj gönderiminin başarılı olup olmadığını kontrol etmek için kullanılmıştır. Bu akış sayesinde lamba veya fanın açık-kapalı durum değişikliklerinde kullanıcıya Telegram üzerinden bilgilendirme mesajı gönderilebilmiştir.

Genel olarak Node-RED akış tasarımı; sensör verilerinin alınması, cihazların kontrol edilmesi, verilerin Dashboard üzerinde gösterilmesi, ThingSpeak platformuna aktarılması ve Telegram üzerinden bildirim gönderilmesi işlemlerini tek bir yapı altında toplamıştır. MQTT giriş ve çıkış düğümleri ESP32 ile haberleşmeyi sağlarken, function düğümleri gelen verilerin işlenmesi ve uygun formata dönüştürülmesinde kullanılmıştır. HTTP request düğümleri ise ThingSpeak ve Telegram gibi harici servislerle iletişim kurulmasını sağlamıştır. Bu yapı, sistemin yalnızca yerel bir kontrol devresi olmasının ötesine geçmesini sağlamış ve projeyi merkezi olarak yönetilebilen, izlenebilen ve bildirim gönderebilen bütünleşik bir IoT uygulamasına dönüştürmüştür.

5.4. Deneysel Sonuçlar

Bu bölümde, geliştirilen IoT tabanlı akıllı priz ve ortam izleme sisteminin çalışma durumu deneysel olarak incelenmiştir. Sistem testlerinde lamba ve fan kontrolü, sıcaklık-nem ölçümü, ortam ışık seviyesi algılama, Node-RED Dashboard üzerinden kontrol, ThingSpeak veri izleme ve Telegram bildirim özellikleri ayrı ayrı değerlendirilmiştir. Deneysel çalışmalar sonucunda sistemin hem yerel kontrol elemanlarıyla hem de uzaktan erişim arayüzleriyle çalışabildiği gözlemlenmiştir.

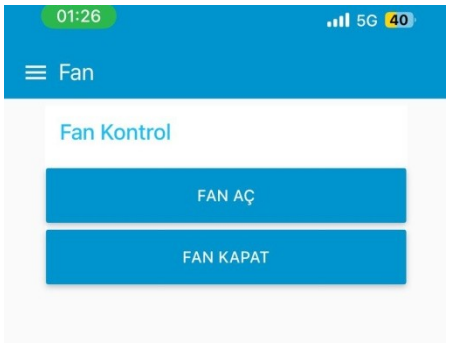
İlk olarak Node-RED Dashboard üzerinden lamba kontrolü test edilmiştir. Dashboard arayüzünde oluşturulan "LAMBA AÇ" ve "LAMBA KAPAT" butonları ile ESP32'ye MQTT komutları gönderilmiş, gelen komutlara bağlı olarak lamba rölesinin açılıp kapandığı görülmüştür. Bu test sonucunda Node-RED üzerinden gönderilen kontrol komutlarının ESP32 tarafından doğru şekilde algılandığı ve lamba çıkışının başarılı şekilde kontrol edildiği doğrulanmıştır.



Şekil 5.4.1 Node-RED Dashboard üzerinden lamba kontrolü

Fan kontrol testi de Node-RED Dashboard üzerinden gerçekleştirilmiştir. Dashboard üzerinde oluşturulan "FAN AÇ" ve "FAN KAPAT" butonları kullanılarak MQTT üzerinden ESP32'ye komut gönderilmiştir. ESP32, gelen komutları işleyerek fan rölesini aktif veya pasif duruma

getirmiştir. Bu test sonucunda fanın web tabanlı arayüz üzerinden başarılı bir şekilde kontrol edilebildiği görülmüştür.



Şekil 5.4.2 Node-RED Dashboard üzerinden fan kontrolü

Sistemde kullanılan DHT11 sıcaklık ve nem sensörünün çalışması da Node-RED Dashboard üzerinden test edilmiştir. ESP32 tarafından okunan sıcaklık ve nem verileri MQTT aracılığıyla Node-RED'e aktarılmış ve Dashboard üzerinde gösterge bileşenleri ile görüntülenmiştir. Test sırasında sıcaklık değerinin 24,6 °C, nem değerinin ise yaklaşık %70 seviyesinde olduğu görülmüştür. Bu sonuç, DHT11 sensöründen alınan verilerin Node-RED arayüzüne doğru şekilde aktarıldığını göstermektedir.



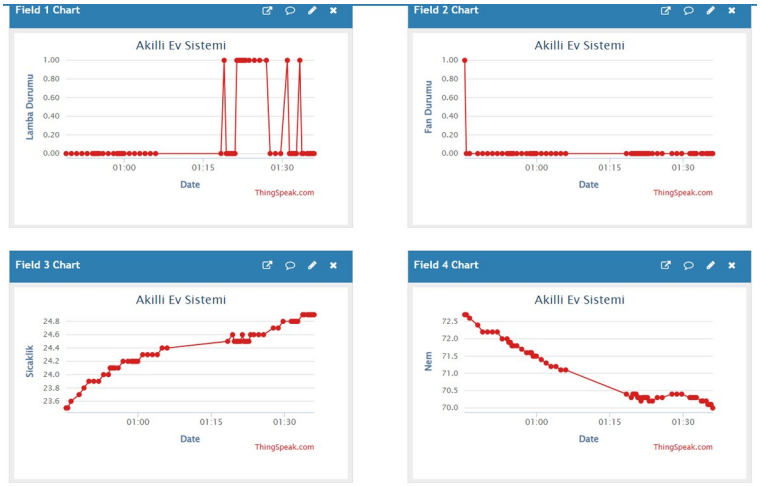
Şekil 5.4.3 Node-RED Dashboard üzerinde sıcaklık ve nem verilerinin gösterimi

LDR sensör modülü kullanılarak ortamın aydınlık ve karanlık durumu test edilmiştir. ESP32 tarafından okunan LDR verisi MQTT üzerinden Node-RED'e gönderilmiş ve Dashboard üzerinde "AYDINLIK" veya "KARANLIK" bilgisi olarak gösterilmiştir. Bu işlemde sayısal olarak alınan 0 ve 1 değerleri, Node-RED üzerindeki function düğümü ile kullanıcı tarafından anlaşılabilir metinsel ifadelere dönüştürülmüştür. Test sonucunda ortam ışığına bağlı olarak Dashboard üzerinde ışık durumunun doğru şekilde değiştiği gözlemlenmiştir.



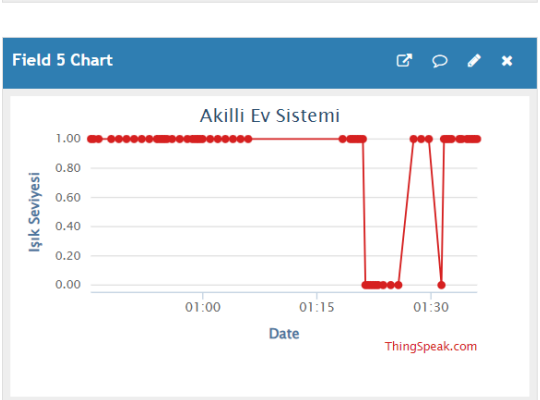
Şekil 5.4.4. Node-RED Dashboard üzerinde LDR ışık durumu gösterimi

Sistemin bulut tabanlı veri izleme özelliği ThingSpeak platformu üzerinden test edilmiştir. Node-RED üzerinden belirli aralıklarla gönderilen lamba durumu, fan durumu, sıcaklık, nem ve ışık seviyesi bilgileri ThingSpeak kanalı üzerinde grafiksel olarak izlenmiştir. ThingSpeak grafiklerinde sıcaklık ve nem değerlerinin zaman içerisindeki değişimi, lamba ve fan durumlarının ise 0-1 mantığında açık-kapalı olarak kaydedildiği görülmüştür. Bu sonuçlar, sistemden elde edilen verilerin yalnızca anlık olarak değil, zamana bağlı olarak da takip edilebildiğini göstermektedir.



Şekil 5.4.5. ThingSpeak üzerinde lamba, fan, sıcaklık ve nem verilerinin izlenmesi

Işık seviyesi verisinin ThingSpeak platformuna aktarımı ayrıca test edilmiştir. Bu testte, LDR sensöründen alınan ortam ışık bilgisi ESP32 tarafından okunmuş ve MQTT üzerinden Node-RED'e iletilmiştir. Node-RED üzerinde işlenen bu veri, belirlenen zaman aralıklarında ThingSpeak platformuna gönderilmiş ve grafik üzerinde 0-1 değerleri şeklinde görüntülenmiştir. Grafikte yer alan değerler, ortamın aydınlık veya karanlık olma durumunu temsil etmektedir. Buna göre 0 ve 1 değerlerindeki değişimler, ortam ışık seviyesinde meydana gelen değişikliklerin sistem tarafından algılandığını göstermektedir. Bu sonuç sayesinde LDR sensöründen alınan verilerin hem Node-RED Dashboard üzerinde anlık olarak izlenebildiği hem de ThingSpeak üzerinde zaman eksenli grafik şeklinde kaydedilebildiği doğrulanmıştır. Böylece sistemin yalnızca cihaz kontrolü yapmadığı, aynı zamanda ortam koşullarını takip ederek bulut tabanlı veri izleme işlevini de yerine getirdiği görülmüştür.



Şekil 5.4.6 ThingSpeak üzerinde ışık seviyesi verisinin izlenmesi

Telegram bildirim sistemi de deneysel olarak test edilmiştir. Lamba ve fan durumlarında meydana gelen değişiklikler Node-RED akışı üzerinden Telegram botuna aktarılmıştır. Test sonucunda fan açıldığında “Fan AÇILDI”, fan kapandığında “Fan KAPANDI”, lamba açıldığında “Lamba AÇILDI” ve lamba kapandığında “Lamba KAPANDI” mesajlarının Telegram üzerinden kullanıcıya ulaştığı görülmüştür. Bu sonuç, sistemin yalnızca kontrol ve izleme yapısına sahip olmadığını, aynı zamanda kullanıcıya anlık bildirim gönderebildiğini de göstermektedir.



Şekil 5.4.7 Telegram üzerinden lamba ve fan durum bildirimleri

Sistemin fiziksel çalışma durumu, proje kutusu üzerinden gerçekleştirilen uygulamalı testler ile değerlendirilmiştir. Bu testlerde özellikle yazılım tarafında oluşturulan kontrol komutlarının fiziksel donanım üzerinde doğru karşılık bulup bulmadığı incelenmiştir. Lamba açık konuma getirildiğinde, kutu üzerinde bulunan lamba çıkışının aktif hâle geldiği ve lambanın sorunsuz şekilde çalıştığı gözlemlenmiştir. Bu durum, Node-RED Dashboard, MQTT haberleşmesi ve ESP32 yazılımı arasındaki veri aktarımının başarılı şekilde gerçekleştiğini göstermektedir.

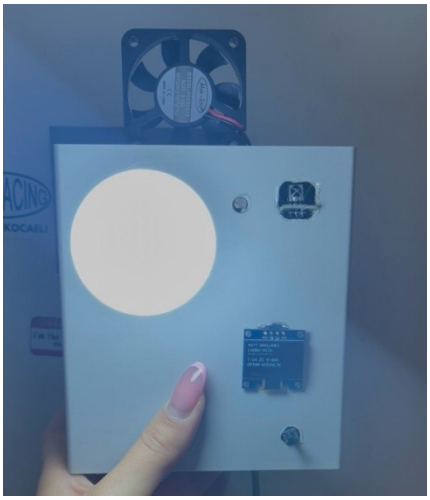
Lamba kontrolü sırasında yalnızca fiziksel çıkışın çalışması değil, sistem durum bilgisinin kullanıcıya doğru şekilde aktarılması da test edilmiştir. Lamba açık konuma alındığında OLED ekran üzerinde MQTT bağlantı durumu, lamba bilgisi, fan bilgisi, sıcaklık, nem ve ortam ışık durumu görüntülenmiştir. Böylece kullanıcının sistemin anlık çalışma durumunu doğrudan cihaz üzerinden takip edebildiği görülmüştür. OLED ekranın bu bilgileri göstermesi, sistemin yalnızca uzaktan kontrol edilebilen bir yapı olmadığını, aynı zamanda yerel olarak da kullanıcıya geri bildirim sağlayabildiğini göstermektedir.

Fiziksel test aşamasında fan, LDR sensör, IR alıcı ve buton gibi diğer donanım bileşenlerinin kutu üzerindeki yerleşimi de değerlendirilmiştir. Fanın kutunun üst kısmında konumlandırılması sayesinde hava akışı sağlanmış, IR alıcı modülün dış yüzeye yakın yerleştirilmesiyle uzaktan kumandadan gelen sinyallerin algılanması kolaylaştırılmıştır. LDR sensörün dış ortam ışığını görebilecek şekilde konumlandırılması, ortamın aydınlık veya

karanlık durumunun doğru şekilde algılanmasına katkı sağlamıştır. Butonun erişilebilir bir noktaya yerleştirilmesi ise kullanıcıya doğrudan manuel kontrol imkânı sunmuştur.

Yapılan testlerde sistemin yazılım ve donanım bileşenlerinin birbiriyle uyumlu şekilde çalıştığı görülmüştür. Node-RED veya mobil kontrol arayüzü üzerinden gönderilen komutlar ESP32 tarafından işlenmiş, ilgili röle çıkışları kontrol edilmiş ve oluşan durum değişiklikleri OLED ekrana yansıtılmıştır. Aynı zamanda Telegram bildirimleri sayesinde lamba ve fan durumlarındaki değişikliklerin kullanıcıya mesaj olarak iletilebildiği doğrulanmıştır. Bu sonuçlar, sistemin kontrol, izleme ve bildirim işlevlerini bütünleşik şekilde yerine getirdiğini göstermektedir.

Genel olarak fiziksel çalışma testi, geliştirilen IoT tabanlı akıllı priz ve ortam izleme sisteminin uygulama aşamasında başarılı şekilde çalıştığını ortaya koymuştur. Yazılım komutlarının fiziksel çıkışlara doğru şekilde aktarılması, sensör verilerinin kullanıcı arayüzlerinde görüntülenmesi, OLED ekranın sistem durumlarını göstermesi ve Telegram bildirimlerinin kullanıcıya ulaşması sistemin temel işlevlerini yerine getirdiğini göstermektedir. Bu nedenle proje, hem yerel hem de uzaktan kontrol edilebilen, sensör verilerini izleyebilen ve kullanıcıya anlık bilgilendirme sağlayabilen bütünleşik bir IoT uygulaması olarak değerlendirilmiştir.



Şekil 5.4.8 Lamba açık durumunda sistemin fiziksel görünümü

Lamba kapalı durumu da ayrıca test edilmiştir. Node-RED Dashboard veya kontrol arayüzü üzerinden kapatma komutu gönderildiğinde, komut MQTT protokolü aracılığıyla ESP32'ye iletilmiş ve ESP32 tarafından ilgili röle çıkışı pasif hâle getirilmiştir. Röle çıkışının pasif duruma geçmesiyle birlikte lamba enerjisi kesilmiş ve lamba kapanmıştır. Bu işlem sırasında OLED ekran üzerinde lamba durum bilgisinin de "kapalı" olarak güncellendiği gözlemlenmiştir. Böylece sistemin yalnızca fiziksel çıkış tarafında değil, kullanıcıya bilgi veren ekran arayüzünde de durum değişikliğini doğru şekilde yansıttığı görülmüştür. Yapılan test sonucunda lamba kontrolünün Node-RED, MQTT haberleşmesi ve ESP32 yazılımı üzerinden senkronize biçimde çalıştığı doğrulanmıştır.



Şekil 5.4.8 Lamba kapalı durumunda sistemin fiziksel görünümü

Genel olarak yapılan deneysel testler sonucunda sistemin lamba ve fan kontrolünü başarılı şekilde gerçekleştirdiği, DHT11 ve LDR sensörlerinden alınan verileri okuyabildiği, bu verileri Node-RED Dashboard ve ThingSpeak platformlarında gösterebildiği belirlenmiştir. Ayrıca Telegram bildirim sistemi sayesinde cihaz durumlarındaki değişikliklerin kullanıcıya anlık olarak iletilebildiği görülmüştür. Elde edilen sonuçlar, geliştirilen sistemin hem yerel hem de uzaktan kontrol edilebilen, sensör verilerini izleyebilen ve kullanıcıya bildirim gönderebilen bütünleşik bir IoT uygulaması olarak çalıştığını göstermektedir.

6.SONUÇLAR VE ÖNERİLER

Bu proje kapsamında IoT tabanlı akıllı priz ve ortam izleme sistemi tasarlanmış, donanım ve yazılım bileşenleri bir araya getirilerek çalışır hâle getirilmiştir. Sistem; ESP32 mikrodenetleyicisi, Raspberry Pi, MQTT protokolü, Node-RED, ThingSpeak, Telegram, DHT11 sıcaklık-nem sensörü, LDR sensör modülü, OLED ekran, IR alıcı, buton, röle modülleri, lamba ve fan bileşenlerinden oluşmaktadır. Projenin temel amacı, elektriksel yüklerin hem yerel hem de uzaktan kontrol edilebildiği, aynı zamanda ortam verilerinin izlenebildiği bütünleşik bir IoT uygulaması geliştirmektir. Yapılan çalışmalar sonucunda lamba ve fanın Node-RED Dashboard, MQTT tabanlı mobil uygulama, IR uzaktan kumanda ve fiziksel buton ile kontrol edilebildiği görülmüştür.

Deneysel testler sonucunda sistemin temel işlevlerini başarılı şekilde yerine getirdiği belirlenmiştir. DHT11 sensörü ile sıcaklık ve nem verileri alınmış, LDR sensör modülü ile ortamın aydınlık veya karanlık durumu algılanmıştır. Elde edilen veriler OLED ekran üzerinde kullanıcıya gösterilmiş, MQTT protokolü aracılığıyla Node-RED ortamına aktarılmış ve ThingSpeak platformunda grafiksel olarak izlenmiştir. Ayrıca lamba ve fan durum değişikliklerinde Telegram üzerinden kullanıcıya bilgilendirme mesajları gönderilmiştir. Bu sonuçlar, sistemin yalnızca cihaz kontrolü yapmadığını, aynı zamanda veri izleme ve kullanıcı bilgilendirme özelliklerini de yerine getirdiğini göstermektedir.

Projenin donanım tarafında, breadboard üzerinde yapılan geçici bağlantılar yerine pertinaks kullanılarak daha düzenli ve kalıcı bir devre yapısı oluşturulmuştur. Devre elemanlarının proje kutusu içerisine yerleştirilmesiyle sistem daha taşınabilir, güvenli ve sunuma uygun bir hâle getirilmiştir. OLED ekran, buton, IR alıcı, LDR sensör, lamba ve fan gibi kullanıcı ile etkileşimde bulunan bileşenlerin kutu dış yüzeyine konumlandırılması, sistemin kullanım kolaylığını artırmıştır. Ayrıca bağlantı noktalarında kısa devre ve gevşeme riskini azaltmak amacıyla bazı kablo uçları ve hassas bağlantı bölgeleri silikon ile sabitlenmiştir.

Yazılım tarafında ESP32, sistemin ana kontrol birimi olarak görev yapmış; sensör verilerinin okunması, rölelerin kontrol edilmesi, OLED ekranın güncellenmesi, IR kumanda ve buton girişlerinin değerlendirilmesi gibi işlemleri gerçekleştirmiştir. Raspberry Pi üzerinde çalışan Mosquitto MQTT Broker ve Node-RED ise sistemin haberleşme, kontrol ve veri yönlendirme altyapısını oluşturmuştur. Node-RED Dashboard sayesinde kullanıcı arayüzü oluşturulmuş, ThingSpeak ile veriler bulut ortamında takip edilmiş ve Telegram entegrasyonu ile sistem durumları kullanıcıya bildirilmiştir. Bu yapı, projenin modüler ve geliştirilebilir bir IoT mimarisine sahip olmasını sağlamıştır.

Geliştirilen sistem, akıllı ev uygulamalarında kullanılabilecek temel bir IoT kontrol ve izleme altyapısı sunmaktadır. Bununla birlikte proje ilerleyen aşamalarda daha da geliştirilebilir. Öncelikle sistemin güvenliği artırılarak MQTT haberleşmesinde kullanıcı adı, şifre ve gerekirse şifreli bağlantı yöntemleri kullanılabilir. Ayrıca lamba ve fan kontrolüne zamanlayıcı, otomatik çalışma senaryoları ve mobil uygulama üzerinden daha gelişmiş kontrol seçenekleri eklenebilir. Fanın yalnızca manuel olarak değil, sıcaklık değerine bağlı olarak otomatik çalışması sağlanabilir. Benzer şekilde LDR sensöründen alınan ışık bilgisine göre lambanın otomatik olarak açılıp kapanması da sisteme eklenebilir.

Bunlara ek olarak sistemde enerji tüketimi ölçümü yapılabilmesi için akım ve gerilim sensörleri kullanılabilir. Böylece lamba ve fan gibi yüklerin çalışma süreleri ve enerji tüketimleri izlenebilir. Proje kutusu daha profesyonel bir tasarımla yeniden düzenlenerek

kablo yönetimi, havalandırma ve güvenlik açısından daha gelişmiş bir yapı elde edilebilir. Ayrıca ThingSpeak dışında farklı bulut platformları veya veri tabanı sistemleri kullanılarak daha kapsamlı veri kaydı ve analiz işlemleri yapılabilir. Sonuç olarak bu proje, IoT tabanlı cihaz kontrolü, ortam izleme, veri aktarımı ve kullanıcı bilgilendirme konularını bir araya getiren başarılı bir uygulama olmuş; ileride geliştirilebilecek akıllı ev sistemleri için temel bir altyapı oluşturmuştur.

KAYNAKÇA

[1] Arduino, “Arduino IDE Documentation,” Arduino Documentation. Erişim tarihi: 25.06.2026.
https://docs.arduino.cc/software/ide/

[2] Espressif Systems, “ESP32-DevKitC Documentation,” Espressif Documentation. Erişim tarihi: 25.06.2026.
https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/index.html

[3] OASIS, “MQTT Version 3.1.1 Standard,” OASIS Open. Erişim tarihi: 25.06.2026.
https://www.oasis-open.org/standard/mqttv3-1-1/

[4] MQTT.org, “MQTT Specifications,” MQTT.org. Erişim tarihi: 25.06.2026.
https://mqtt.org/mqtt-specification/

[5] Eclipse Mosquitto, “Mosquitto Documentation,” Eclipse Mosquitto. Erişim tarihi: 25.06.2026.
https://mosquitto.org/documentation/

[6] Node-RED, “Node-RED Documentation,” Node-RED. Erişim tarihi: 25.06.2026.
https://nodered.org/docs/

[7] MathWorks, “ThingSpeak Documentation,” MathWorks Documentation. Erişim tarihi: 25.06.2026.
https://www.mathworks.com/help/thingspeak/index.html

[8] MathWorks, “ThingSpeak IoT Analytics Platform,” ThingSpeak. Erişim tarihi: 25.06.2026.
https://thingspeak.mathworks.com/

[9] Telegram, “Telegram Bot API,” Telegram Core. Erişim tarihi: 25.06.2026.
https://core.telegram.org/bots/api

[10] Nick O’Leary, “PubSubClient: A Client Library for Arduino MQTT,” GitHub. Erişim tarihi: 25.06.2026.
https://github.com/knolleary/pubsubclient

[11] Arduino-IRremote, “Arduino-IRremote Library,” GitHub. Erişim tarihi: 25.06.2026.
https://github.com/Arduino-IRremote/Arduino-IRremote

[12] Adafruit, “DHT Sensor Library,” GitHub. Erişim tarihi: 25.06.2026.
https://github.com/adafruit/DHT-sensor-library

[13] Aosong Electronics, “DHT11 Digital Relative Humidity and Temperature Sensor Datasheet.” Erişim tarihi: 25.06.2026.
https://www.electronicoscaldas.com/datasheet/DHT11_Aosong.pdf

[14] Adafruit, “Adafruit SSD1306 Arduino Library,” GitHub. Erişim tarihi: 25.06.2026.
https://github.com/adafruit/Adafruit_SSD1306

[15] Adafruit, “Adafruit SSD1306 Class Reference,” Adafruit Documentation. Erişim tarihi: 25.06.2026.
https://adafruit.github.io/Adafruit_SSD1306/html/class_adafruit___s_s_d1306.html